

Battery Management System

For Electric Formula SAE Racecar

Spring Quarter Report

by

Drake Vogelpohl

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
1 Introduction.....	3
1.1 Scope.....	3
1.2 Roadmap.....	3
2 Background.....	5
2.1 NFR25's Accumulator.....	5
2.2 Orion BMS.....	11
2.3 Design History and Flow.....	12
2.3.A Previous Custom BMS.....	12
2.3.B Winter Quarter 2025 scope.....	12
2.3.C Spring Quarter 2025 Scope.....	13
2.3.D Next year and Future Years Scope.....	13
3 Hardware.....	14
3.1 Daughterboard.....	14
3.1.A Cell Depopulation.....	20
3.1.B Sense PCBs, Flex PCBs, and Issues.....	20
3.1.C Layout.....	22
3.1.D List of improvements.....	24
3.2 Mainboard.....	25
3.2.A Layout.....	33
3.2.B List of improvements.....	34
4 Firmware.....	35
4.1 AD BMS (Low Level).....	35
4.1.A List of Improvements.....	42
4.2 NFR BMS (High Level).....	43
4.2. DMA and RTOS.....	48
4.2.B State of Charge Estimation.....	48
4.2.C Ideal NRF BMS Behaviour.....	48
5 Testing and Validation.....	50
5.1 Using the Dev Boards.....	50
5.2 Virtual Comm Port.....	50
5.3 Custom Drivers.....	51
5.4 Cell Balancing.....	52
5.5 Open Wire Check.....	53
5.6 Charging with the Elcon Charger.....	54
5.7 Dynamic Testing.....	55
5.8 Chips Dying.....	56
6 Future Designs.....	57

6.1 RTOS and DMA.....	57
6.2 Pack Monitor.....	57
6.3 Having the BMS as purely a slave.....	58
References.....	59

1 Introduction

This will focus on the hardware and firmware development of a custom Battery Management System (BMS) using the Analog Devices (AD) adbms68 chipset for a Formula Electric SAE vehicle. This will allow us to have higher confidence, better integration, and design of our high voltage battery (accumulator) that powers the vehicle. Projects like this that develop new innovative designs drive the learning and knowledge base present on Northwestern Formula Racing (NFR) and will allow us to continually improve the competitiveness of the overall team. The goal for this report is for it to be referenced by any (NFR member or otherwise) designing any BMS in future years, as well as documentation of work done this year.

NOTE: This report is being written at the end of the school year and is intended to fully document the high-level choices, background, designs, validations, challenges, and upcoming plans of NFR25's BMS.

1.1 Scope

This project was designed and testtested over the course of two quarters (WQ25 and SQ25). This report will focus on where the current designs of the BMS lie at the end of this year, along with plans for future years, and as a result will not go over things that were designed/implemented in WQ25 and have since changed. For a more detailed report of the work done and the state of the BMS in WQ25, see [here](#).

1.2 Roadmap

The report starts with the background section. This explains the motivation for a custom BMS, gives background into the battery and the system that this BMS lives in and interacts with, and goes over the high level design goals.

Next is the hardware section. This goes over the designs of the daughterboards (including how the signals get from the cells to the chips) and the mainboard.

After that is the firmware section. This is broken into two sections. First, the firmware that interacts with the daughterboards, including the drivers and interfacing layer with these drivers. This section is noted as “low level” as interfacing and getting values from the daughterboards is mostly following the command list and memory map from the datasheet rather than performing high (system) level logic. Second, is the section that performs the actions required by the BMS and the rest of the car, with the data collected in the “low level”. As it performs the system level logic, it is noted as “high level”.

The last two sections are the testing and validation section and the future designs section. Even though these sections do not cover designs, the testing and validation section is arguably the most important aspect of this report. The reason for this is all the designs are open sourced and available to look at. While this documentation could be helpful to reference, the designs themselves are fairly self explanatory and this long winded documentation is not *that* useful. On the other hand, understanding how a system as complex as a high voltage battery designed in a way that is very similar to those of every other FSAE team behaves under load is something that is not well documented and not intuitive at all. I hope to shed some light on this through our testing towards the end of the year.

Finally is the future designs section. This is quite self explanatory, but is worth noting the timelines for getting a working BMS this year were *very* rushed, insofar as needing a fully working MVP in less than 3 months after starting this project. For reference, most teams take a year or two to accomplish this task. The need for this rushed timeline is explained more in the background section, but it resulted in many features (including an entire chip to perform pack operations, the ADBMS2950) not making it into this year’s designs, and this section provides insight into what an “ideal” FSAE BMS might look like.

2 Background

Northwestern Formula Racing (NFR) is a student-led organization that designs, builds, manufactures, and tests Formula style cars at Northwestern University. A 600V, 6.8kWh battery pack designed using 4.5Ah lithium-ion cells powers this year's vehicle (NFR25). The battery pack needs a Battery Management System (BMS) to monitor individual cell voltage and to balance voltage differentials that arise due to cell tolerances. Commercial-Off-The-Shelf (COTS) BMSs exist, but they are expensive, large, and not optimized for individual battery packs. Developing a custom semi-distributed BMS offers several advantages, including lower costs, higher measurement accuracy, and simpler wiring achieved by placing the measuring device close to the monitored cells. This project will design, implement, and test a BMS specific to NFR's battery pack using Analog Devices' BMS chipset and an STM32 microcontroller.

2.1 NFR25's Accumulator

Fundamentally, the battery management system has to manage a battery. This battery is the HV battery, also called the accumulator, we use to power our car and is made up of 420 Molicel P45Bs, 21700 4.5Ah lithium-ion cells, in a 140 series 3 parallel configuration. This results in a peak voltage of 588V, nominal of 504V, and undervoltage of 350V. It also allows us to discharge 135A continuous, ~80 kW at full charge (neglecting thermal throttling and cutoff). A good understanding of the design of the accumulator is required to design a well integrated BMS.

NFR25's accumulator was designed from the bottom up, as its main goal is to package the 420 cells into as space efficient an area as possible while still maintaining proper electrical safety and thermal management. The decision of the size of our battery pack was determined solely through extensive simulation as we have not had a previous driving car to compare real world data to the simulation data. The setup and running of the simulations as well as how this was determined to be the optimal size is outside the scope of this report but one of the most important engineering decisions to be made when designing an accumulator.

Dictated in the FSAE rules and following standard OEM best practices, the cells are separated into modules called segments, and kept separate from the wiring/electrical compartment that

holds things such as insulation monitors and isolation relays. Following our team nomenclature, this compartment is known as the penthouse. Both the segments and the penthouse are enclosed in the accumulator container (Figure 2). The container is made from $\frac{3}{8}$ " sheet aluminum laser cut and bended by our sponsor, [Bystronic](#). The bent aluminum and cut internal walls are then riveted in place, and the internal faces are covered with Nomex 410 to electrically insulate and properly fireproof them as outlined in the FSAE rules. Finally the mounts are glued onto the side of the accumulator using a structural adhesive in effort to avoid any weld beds on the accumulator. The specifics of the designs are left out for brevity.

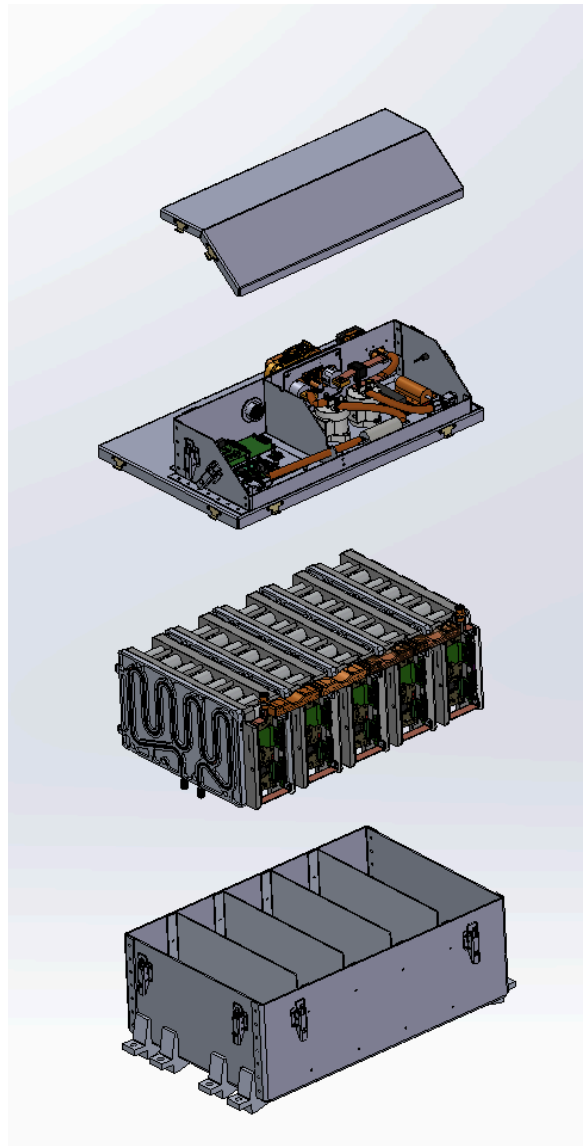


Figure 1: NFR25's Accumulator in Exploded View

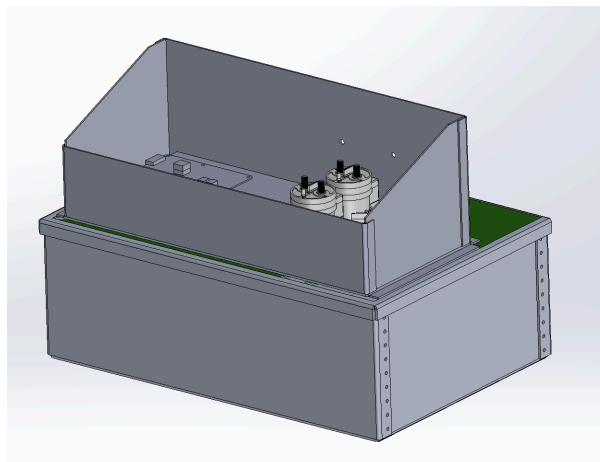


Figure 2: Accumulator Container

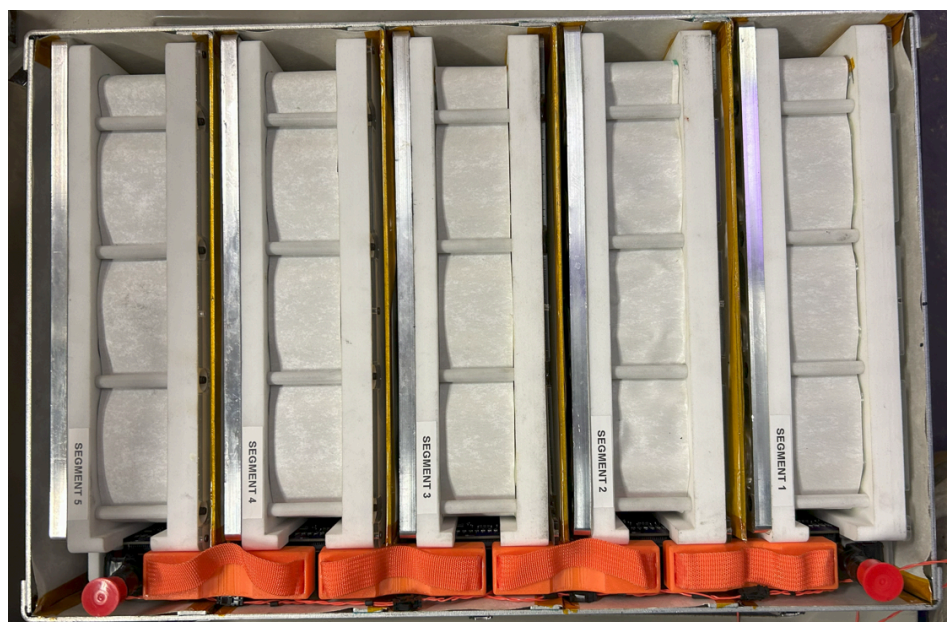


Figure 3: Segments in Accumulator Container with Maintenance Plugs Installed

There are 5 identical segments with 84 cells per segment that are wired in series to complete the full pack. Each segment has 28 series groups of 3 cells in parallel. The number of segments and how each segment is designed is again outside the scope of this report, but it was done such to minimize the space given by the frame and allow the cells to be as low and towards the center of the car as possible. This is due to the fact that the cells are one of the heaviest and descents components in the car and a low and in the center of the wheelbase center of gravity is ideal for vehicle dynamics.



Figure 4: Close up picture of rim bonding and bolted busbar connections

Omitting all of the design details and just accepting the design of the segments as is, the segments are 6 cells tall and 14 cells long, packaged in a hex pattern to minimize the area. Each segment has a top cell holder face and a bottom cell holder that constrain the cells and are made out of SLS 3D printed nylon. The cells are then glued into the cell holders via DP460. The top cell holder allows for the top of the cell to be exposed as well as a small section of the rim. The rim is then used as the negative terminal of the cell and wire bonded in a fashion known as “rim bonding” (Figure 4). This allows us to connect every cell while all having them point in the same direction. We do this so that the negative tab of the cell (what would normally be where you would connect/access the negative of the cell) is exposed, allowing for negative tab cooling, i.e. water cooling via a cold plate on the bottoms of the cells. In order to do this, the negative cell holder leaves about 1 mm of the bottoms of the cells exposed and then an aluminum, custom designed and manufactured cold plates are bolted into the bottom of the cell holders with an electrically resistive thermal pad interfacing between the segments and cold plates.

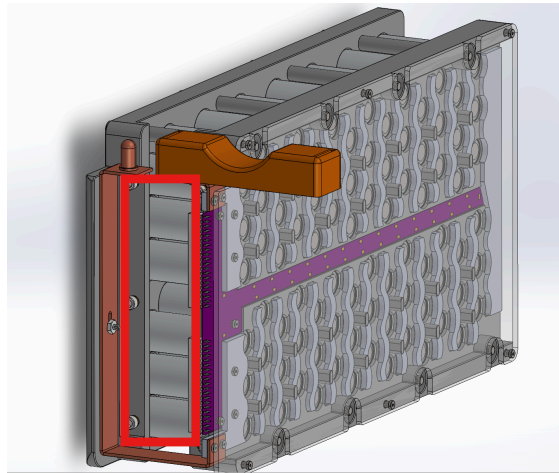


Figure 5: CAD model of a Segment Highlighting Where Custom BMS Sits in Red.

Each segment is connected to each other via copper busbars that run along the top and bottom cell holders and maintenance plugs (positive locking, tool-less removable plugs that can connect/disconnect segments), as shown in copper and orange in Figure 5. Maintenance plugs as a means to connect/disconnect segments are required in FSAE rules to be positive locking and tool-less removable, we achieve this by designing a custom plug that mate with a pin on the copper bus bar and are constrained by the accumulator lid.

Because of our segment configuration, and vertical height being more important than horizontal, it was determined to place the BMS daughterboards where the busbars were already creating dead space, as shown in the red box in Figure 5. In order to get the voltage and temperature signal lines to the daughter boards, sense and flex PCBs were designed. These act as glorified wires. The sense PCB is the purple T-shaped PCB in Figure 5. There are wire bonds from the busbars to the sense PCBs that we covered in DP460 for rigidity (Figure 4) that properly fuse the sense lines. The thermistors are glued to the sides of evenly spaced cells throughout the segments before being assembled into segments and the leads of these thermistors are soldered to pads on the sense PCB. Because the sense PCB runs along the face of the segments and to have the daughterboards not increase the area they have to be perpendicular, there needs to be some sort of connector between the two. Having this connection be wires means you have to crimp that many wires and lose a lot of the benefits over a COTS BMS and using mezzanine connectors

would make the daughterboard stick out. To get around these issues we design a custom flexible PCB, the yellow PCBs with high and low on them in Figure 6. Again, these flex PCBs act as custom ribbon cables routing signals from the sense PCBs to the daughterboards.



Figure 6: Face of the Daughterboard Installed in a Segment

Sitting on top of where the segments are located, is the penthouse (electrical and wiring compartment). This is still a part of the accumulator and is where the BMS mainboard lives. In this box is also the shunt resistor to provide a current measurement. An insulation monitoring device to monitor the insulation between the tractive system and the grounded low voltage (chassis) system. Isolation relays (including a precharge relay) to keep the always energized tractive system from leaving the accumulator unless the relays are closed. There are also smaller items such as a HV indicator LED that turns on whenever there is voltage present after the contactors and other miscellaneous items.

2.2 Orion BMS

The most popular COTS BMS used by FSAE teams is the Orion 2 BMS. The stand-alone component costs around \$1,500 for the one that can measure 144 cells and can cost up to \$2,300 including the current sensor and harness. Not only does this cost a lot, but it is also a centralized system. This is where the real pain occurs because teams using this BMS have to route every single cell voltage line to this centralized location, resulting in a harness nightmare displayed in figure 7.



Figure 7: Orion BMS Harness

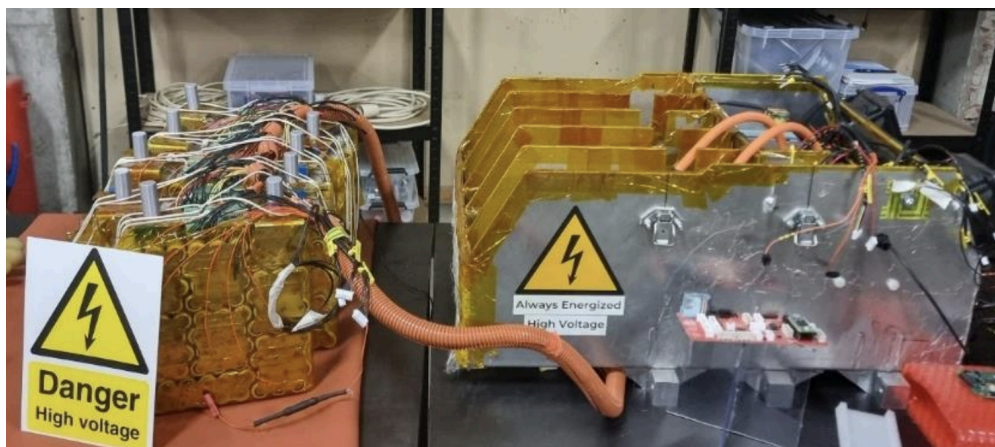


Figure 8: UGent Racing 2022 segments and accumulator container

Figure 8 is what using an Orion BMS for an FSAE battery could look like. This is UGent Racing's battery from 2022. Note the mass of wires coming out and eventually feeding into the orange conduit. These are their voltage and temperature sense lines that are being routed to the centralized Orion BMS in the accumulator. Even though when the segments are placed into the

accumulator a lot of this wiring will be hidden, it still shows a lot of the pain points of using a COTS BMS.

2.3 Design History and Flow

2.3.A Previous Custom BMS

In previous years there was a custom BMS designed on top of the Texas Instruments' battery management platform chips, namely the BQ76952. There were a lot of things this system did right such as the high level semi-distributed system and use of sense PCBs. However, there were a number of issues with the design itself. Namely the chips would die seemingly randomly and drain the entire segment they were on. Furthermore the code itself was very bloated and hard to use, as well as not being well understood or documented. This coupled with small errors in the drivers in open wire check and a watchdog that would restart the chip, provided the justification to create a completely new custom BMS.

The major change was made to switch to the Analog Devices' ADBMS platform, as this is more similar to the chips used by major OEMs, as well as a sponsorship from Analog Devices. On top of that, the decision was made to move away from a Teensy 4.1 as the mcu to an STM32F405. This was because STM32's have significantly more documentation, support, and are actually used in industry as opposed to a dev kit. Apart from the high level design goals of using a semi-distributed system with daughterboards on the segments and sense PCBs to help route the signals to the daughterboards and making the FSM states and CAN messages backwards compatible, the previous BMS was not referenced at all in this design. This was because the previous BMS did a lot of things, especially in firmware, in a convoluted manner and for the sake of wanting to design something entirely on our own.

2.3.B Winter Quarter 2025 scope

The work of this year was split into two different quarters. Winter quarter 2025 (WQ25) was focused on understanding the hardware, the initial PCB designs, and the initial firmware. The goal was to make an MVP that could be used in our current accumulator. Given the short length of each quarter, sacrifices had to be made in order to meet this deadline and we plan on

addressing these sacrifices next quarter. Furthermore, this had to be designed as a complete “drop-in-replacement” for our current BMS, placing further restriction on the operation of the BMS, mainly on interactions with the rest of the car.

2.3.C Spring Quarter 2025 Scope

This quarter has been focused on validation, and addressing issues as they come up during validation. Namely, redoing sections of the hardware to increase functionality such as being able to measure current in both directions and writing my own drivers (more on this later). This quarter has also raised questions that we have not answered and would like to look into in future years. This validation has been very interesting and important as it is the first time we have gotten dynamic data from the BMS and understanding how the battery behaves under a dynamic load both charging and discharging.

2.3.D Next year and Future Years Scope

Next year we would like to dive into addressing the issues that were brought up during validation this past quarter as well as addressing the high level sacrifices made this year to meet the aggressive timeline.

3 Hardware

The hardware is broken into two sections based on the two different PCBs that were designed. The “daughterboard” is the PCB that lives on each segment and does the actual voltage and temperature measurements as well as balancing cells when necessary. The “mainboard” lives in the penthouse (top section of the accumulator, above the segments) and communicates with the daughterboards, communicates with the rest of the car, controls the isolation relays, and measures the current flowing into/out of the accumulator.

3.1 Daughterboard

One daughterboard will live on each segment and contains two ADBMS6830 chips to perform 28 series voltage measurements and 16 temperature measurements. This board is constrained to the face of the segments as described in the background section and has galvanic isolation on the communication in/out of the board. The daughterboard gets the signal lines through a sense PCB that is electrically connected and fused to each of the series cells. There is a connector on that PCB and a corresponding connector on the daughterboard that will be connected via a flex PCB acting as a ribbon cable. The daughterboard is designed according to best practices and the adbms6830 datasheet and functional block diagram (Figure 9) [1].

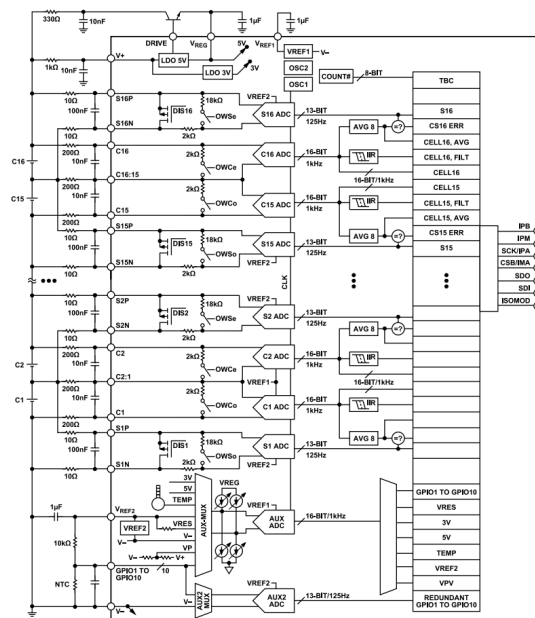


Figure 9: Functional Block Diagram for adbms6830B

Each daughterboard has the following sections of hardware: voltage sensing, temperature sensing, cell balancing, chip power, connectors, and communication lines.

Voltage Sensing is done by routing each series cell connection to the BMS. This is done by “sense” and “flex” PCBs described in more detail later. What is important to know is that the voltage sensing is differential on this chip. This means that if each of the 14 cells that a chip sees is at 4.2V there would be a voltage differential of 58.8V between the negative terminal of the bottom cell and the positive terminal of the top cell of this chip. However, each cell has its *own* channel and corresponding 16-bit Sigma-Delta ADC. It is interesting to note that other BMS chips have different ADC configurations. For example, Texas Instruments' BQ BMS chipset uses only one 24-bit Successive Approximation ADC that is muxed between each cell. The voltage sensing hardware is fairly straightforward, just a 200 Ohm resistor in series and a 10nF differential capacitor between sensing lines as shown in Figure 10, where Cellx is the voltage signal line input for cell x and Cx is to the corresponding pin on the chip.

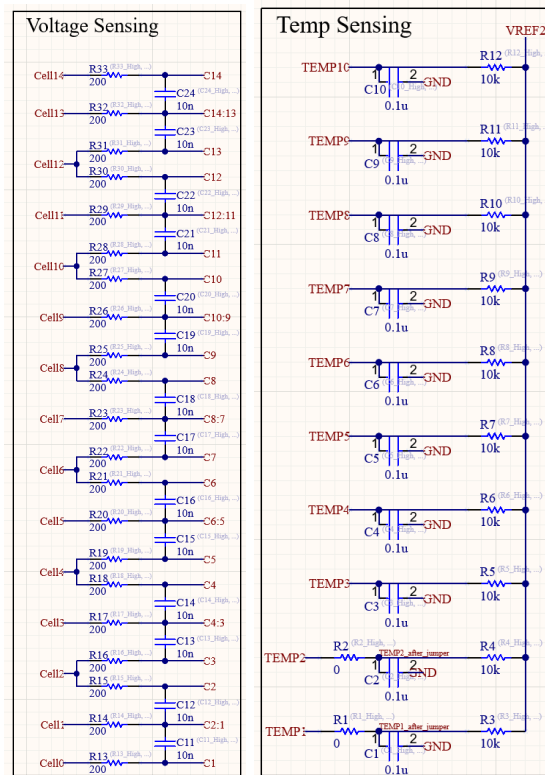


Figure 10: Schematic of Voltage and temperature sensing

Temperature sensing is done by a voltage divider between an on-board resistor and a thermistor on the segment. The chip provides a regulated reference voltage of 3V to be used for this purpose. Our negative temperature coefficient (NTC) thermistors are glued with thermally conductive, electrically insulative epoxy to the outside of cells evenly spaced throughout the pack. We decided to use 10k Ohm thermistors with long leads so that we can easily solder them from the cells to the sense PCBs. As shown in Figure 10, there are 10k Ohm resistors from Vref2 to the corresponding thermistors in the pack. A decoupling 100nF capacitor across the thermistor because the daughter board is mounted on the segment and sees high EMI. The chip then measures the voltage across the thermistor in reference to V-.

Voltage sensing is done on the C channel where each voltage has a dedicated pin of Cx. Cell balancing is done on a second, S, channel. There is a dedicated positive and negative pin for each cell labeled as SxP or SxN where x is the cell. This chip supports passive cell balancing where excess charge can be sloughed off via shorting a resistor across the cell. This can be done via the internal FETs that support up to 300mA of current, or external FETs that can be sized according to cell balancing needs and controlled via the internal FETs. We decided the internal FETs current would suffice. To get near this current we use 20 Ohm cell balancing resistors. Ideally two 10 Ohm resistors would be on either side of the internal switch to limit the heat dissipation each resistor sees. However, an LEDs was wanted across the cell balancing resistor to give a strong visual when cell balancing was occurring, and the forward voltage of the LED throughout the voltage range of the cell (2.5V-4.2V) would not be possible if two resistors were used.

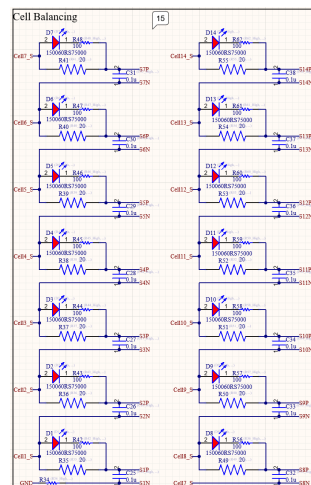


Figure 11: Cell Balancing Schematic

Due to reading highly sensitive analog voltage lines and needing to cell balance and power the chip on the same lines, kelvin connections are required. The sensing kelvin connections would ideally be broken off at the busbars, but due to needing thin wire bonds and not wanting to have twice as many potential points of failure, they are broken off at the sense PCB pads. This provides a path for current, drawn by powering the chip and cell balancing, that is separate from the path that sensing is done on. The purpose of this is while the resistance in the lines should be quite low and the current is also low, the signals are so sensitive that they still need to be separated. This is even more important when the sense lines are very long as is the case for our battery pack, causing there to be very non-negligible resistance. The voltage drop across the sense PCB traces will be more important later.

The chip powers itself directly from the cells it is measuring. This also means it is always powered and thus can kill your battery. Luckily, the chip is only fully on and drawing the max current, $<30\text{mA}$, when actively being communicated with. When it is not, e.g. the accumulator is out of the car and the mainboard is not powered, the chips only draw a few micro amps of current, hardly enough to kill a large battery such as the accumulator unless left for thousands of days unattended. The chip powers itself through an internal LDO or low drop out linear regulator. It functions by changing the effective resistance of an external BJT to provide a constant voltage at the different current draws of the chip. Decoupling capacitors were placed as well as a ferrite bead to help maintain a constant voltage. Finally an LED was placed in series such that it will light up whenever the chip is not in sleep mode.

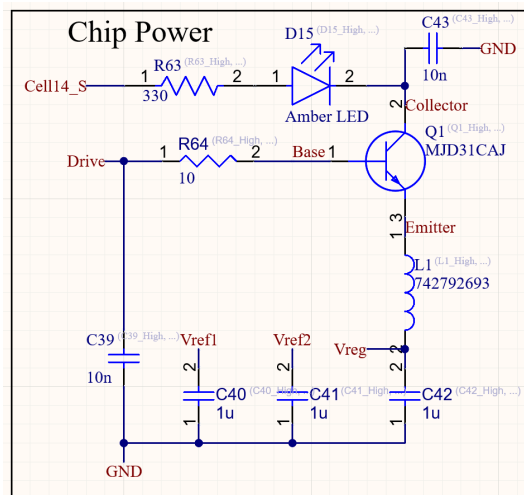


Figure 12: External LDO circuitry

The connectors for the daughterboard were chosen to meet the voltage requirement of each segment. The ones we chose have SMD pads and the header pins on the daughterboard and corresponding connector on the flex PCB to shield the sense lines from accidental shorts as much as possible. The problem with these high pin count connectors (ours are 40-pin) is they are hard to remove but don't have positive locking. They are difficult enough to remove such that they "shouldn't" come off while driving, but that has the added annoyance of being quite difficult to remove and sometimes rip the flex PCB when removing. A solution to look into for future years are zero insertion force, or ZIF, connectors with a positive locking feature. This connector has a couple of benefits. Mainly, as implied by the name it is very easy to install the flat flex cable and is locked such that it wouldn't be loosened by the vibrations in an automotive environment. Secondly, we already use flex PCBs to route the signals from the sense PCBs to the daughterboards. This means that we would only have to buy one connector (the ZIF) instead of two (one connector and corresponding header pins), and that we would no longer have to solder to the flex PCB, cutting down on complexity and cost. The only fear with ZIF connectors is that it feels it is possible to misalign the pins when inserting due to the small pitch and no physical pins to align anything. If adjacent pins are misaligned or shorted together, this would cause no harm in a normal setting when everything is powered off when plugging in. However, in this use case the batteries are always energized so a misalignment or short would be detrimental causing a dead short on the cells and blowing the fusing. While the fusing is there for this exact reason, it is very difficult to replace and something that ideally would never be tested. For this reason, more testing via a PCB with this connector and a corresponding flex PCB would be required to prove that it will never short when inserting before it is used with cells.

Finally the communication lines. The comm lines (Figure 13) are the least straightforward aspect of this PCB. The datasheet does not give very many helpful references for how to design the circuitry for them. What is known is that there needs to be resistors tying the differential lines together, series resistance in the lines, decoupling capacitors to help with noise, TVS diodes to prevent voltage differentials killing the chip when plugging the connector in, and some form of galvanic isolation to isolate each chip in the pack from each other and the mainboard. To start, the datasheet provides a reference for 22nF of series capacitance if using capacitors for galvanic isolation, and the TVS diodes are just standard used for this purpose. Looking into similar

3.1.A Cell Depopulation

Because each chip has the capability to measure 16 cells and we only want each chip to measure 14 cells due to the accumulator architecture, we have to depopulate the top channels by tying them to the top cell via a resistor as described in the datasheet and shown in Figure 14. Note that we are depopulating an even number of cells, and there is different hardware for depopulating an odd number, again outlined in the datasheet.

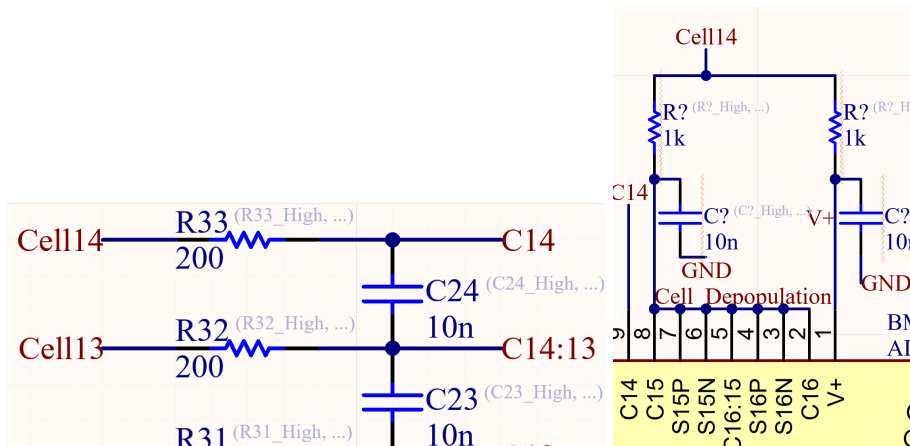


Figure 14: Cell Depopulation Schematics

3.1.B Sense PCBs, Flex PCBs, and Issues

While these were not designed by me (designed courtesy of Anton Walvoord), they are integral to the design of the BMS. We have sense PCBs that are glued to the face of each segment. The PCB has exposed pads near each busbar and are connected to the busbars via a ~1A fusing wire bond. Wire bonding for fusing is not required, but the accumulator requires cell wire bonding to do negative tab watercooling. As a result, we have a wire bonding sponsor, Hesse Mechatronics, who was willing to do the sense wire bonds at the same time as the cell wirebonds. At the top of the sense PCB is a header. To get from the sense PCB header to the BMS is the “flex” PCB. This is a flexible PCB with two headers that acts as a glorified ribbon cable. The top section sense PCBs is displayed in Figure 15 for reference.

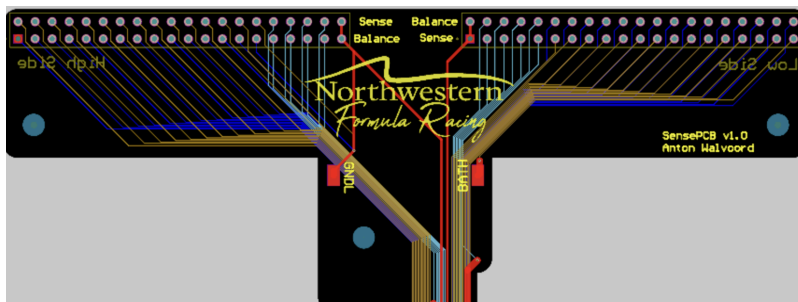


Figure 15: Top section of the Sense PCB

Keen readers might have noticed how thin the traces are in Figure 15. They are the JLCPCB minimum width of 0.1mm. Because these are quite long PCBs, the longest trace has a resistance of ~ 1.5 Ohms. While this doesn't sound significant because the sensing kelvin connection preserves the sensitive voltage signal, it is significant. When cell balancing with maximum current, there is too much of a voltage drop over the cell balancing lines such that the sense lines go below the minimum voltage in reference to the V- or chip ground where $C_x > -0.3V$ in reference to V-. In other words the sense lines are still the at desired voltage of the busbars due to the kelvin connection, but because of the voltage drop of when balancing at maximum current, the V- is artificially greater than the busbar voltage by more than 0.3V. On top of this issue the pads that get wire bonded to were labeled backwards, i.e. cell1 was labeled as cell14 and vice versa. Unfortunately, due to not enough people checking over it, the sense PCBs were fabricated and glued to the segments before these issues were caught.

These issues were eventually caught and solutions were as follows: flipping the sense lines in the flex PCB to fix the backwards labeling issue. The voltage drop issue was solved by balancing at a lower current determined by estimating the resistance and the maximum voltage drop.

Unfortunately the lower cell balancing current means charging will take longer, and the cell balancing LEDs had to be taken off.

After empirically testing charging and discharging, the cells are close enough such that the low cell balancing current, 36mA nominal from the 100 Ohm resistance, is not as significant of an issue as first thought. It still has to cell balance a little longer than just straight charging if the cells are desired to be very close in voltage, but the current cell balancing configuration is easily able to maintain a balance of $<15mV$ from the max cell to the min cell after a charging. It is also

worth noting that because of the internal resistance of the cells and the resistance of the wirebond, there is a corresponding voltage drop when performing cell balancing resulting in a lower bound of balancing that increases as the cell balancing current is increased. I.e. there is a system level tradeoff between how quickly cells balance with the time it takes to balance. Also when performing a cell balancing test on the bench with the original 20 Ohm resistors and LEDs the chips got very warm, and potentially might have run into thermal issues. Resulting in the cell balancing resistance to not be as trivial of a decision as originally thought, and higher resistance than the original 20 Ohms but lower than the 100 Ohms for this year might make the most sense.

3.1.C Layout

The size of each daughterboard is predetermined by the face of the segments it lives on as shown in Figure 16. After that the PCB is broken into two halves, one for each chip. With only the galvanic isolation caps crossing over this gap. On the top layer, (Figure 16) there are the voltage sensing lines (along with associated filter networks), LDO circuitry, and communication circuitry (both in and out) all leading to the chip in the center. The differential capacitors across the sensing lines are placed as close as possible to the chip, as well as the external capacitors that are required by the chip. The power (LDO) circuitry is placed around the outside so as to not interfere with the analog signals, along with cell balancing resistors and the temperature sensing as these lines are not as sensitive. Underneath the sensing lines is an uninterrupted ground pour to provide good return paths for the signals. The component placement was done in such a way to make it easier to assemble by having many components in a line (apart from the differential caps that were placed as close as possible to the chip and the comms circuitry). The digital communication circuitry was placed such that it was away from the sensitive analog signals and allowed for the lines to route as close together as possible because they are a differential pair. The ground plane does not fully extend under the comms line to not be under the galvanic isolation and ideally a differential pair is its own return path. However, the ground pour could be done better such that there are no impedance discontinuities under the communication lines that could potentially cause ringing. The position of the communication connectors is also important as the original design had them on opposite sides resulting in the high side comms connector being covered by the flex PCB, shown in Figure 17. This was solved quite trivially by routing the comms connectors to be on the same side and not covered as shown in Figure 16.

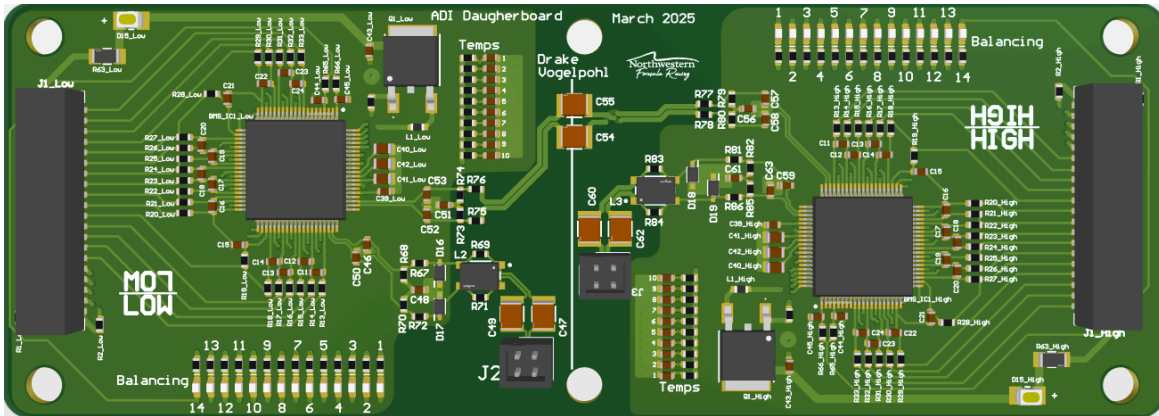


Figure 16: Top 3D view of Daughterboard Layout

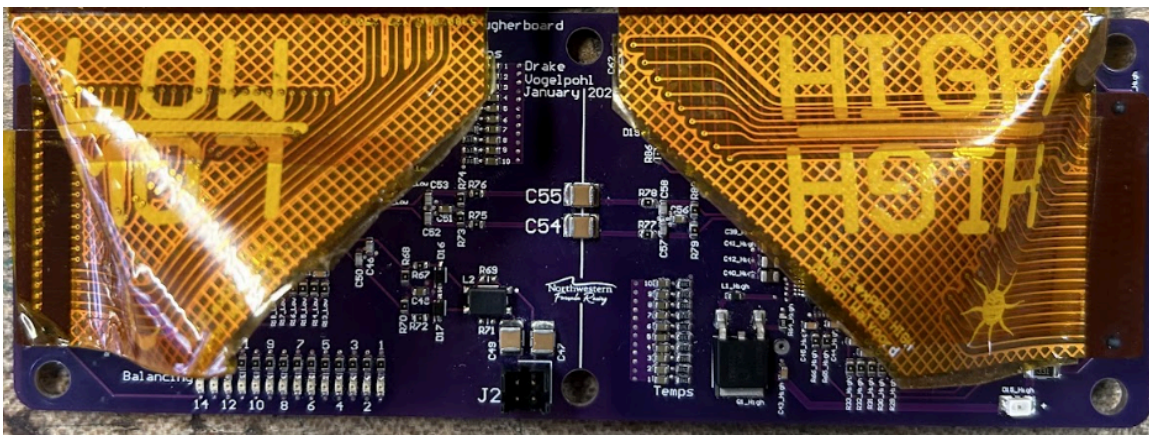


Figure 17: Daughterboard with Flex PCB Attached

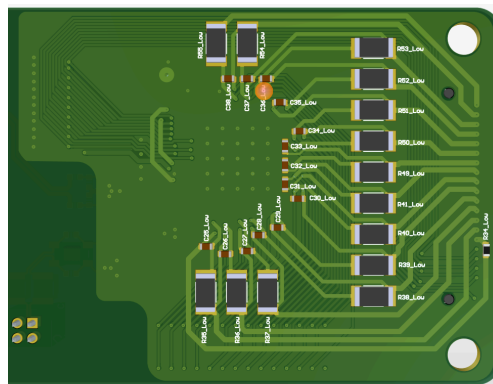


Figure 18: Bottom 3D view of Daughterboard Layout

On the bottom layer is just the cell balancing resistors and their associated differential caps (placed right under their corresponding pads). The purpose of them being on the bottom layer is to make the top layer with the sense lines less crowded. Also cell balancing is switching on and off with noisy edges that can potentially affect the sense lines. For this reason they are placed on

the other side of the board with many ground layers of separation. These resistors also can generate a decent amount of heat. When fully balancing with 20 Ohms resistors, there can be up to 0.882W of power per resistor. This heat is not ideal to have right next to the chip and sensing resistors whose value will slightly shift due to temperature.

After learning more about PCB layout and looking at this design more there are some adjustments I would like to make. I would like to have this board be a 6-layer board. Currently the third plane is not strictly a ground pour, but also has the cell balancing LED and thermistor traces on it. This is not ideal as it compromises the return path for both these signals and the cell balancing signals. The decision to make this a 4-layer board was actively made to save cost with the knowledge that it would lead to decreased integrity as mentioned above. The idea is these signals are not very important and it wasn't worth it to move to a 6-layer board, but it would be interesting to see the design as a 6-layer board.

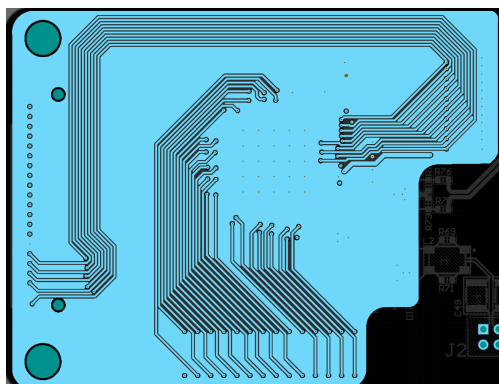


Figure 19: Daughterboard layer 3

3.1.D List of improvements

- ZIF connector as described above
- Make the ground pour extend completely under isoSpi leading up to the galvanic caps
- Make it a 6 layer (or more board) so that:
 - No traces have to be routed on plane layers
 - The communication lines can be shielded on the top and bottom by ground layers
 - Star grounding could be used to isolate the sensitive voltage sensing signals ground from the rest of the board and connect it only on lower layers

- I.e. Split the ground pours to protect the analog signals and connect on lower layers at single points with vias
- Signal integrity simulations
 - This would be nice to learn how to do and to validate the current designs susceptibility to external interference
- Fix issue with chips dying (more on this issue in the validation section)

3.2 Mainboard

The mainboard sits in the wiring compartment (Figure 1) and contains an STM32F4 microcontroller and the ADBMS6822 communication chip. This board also contains an isolated amplifier to measure pack current via the pack shunt resistor and circuitry to drive the isolation relays. The mainboard talks with the daughterboards to gather cell data, evaluate fault states, and communicate this with the rest of the car over a Controller Area Network (CAN) bus. The firmware lives on the STM32F4 and executes the functionality described above.

The mainboard is a different complexity compared to the daughterboard. The daughterboard has fairly straightforward circuitry dictated by the datasheet. That being said there is a lot to talk about a simple capacitor network for example for the daughterboard due to describing how it interacts with the ADBMS6830B chip. The layout is also much more interesting as well for the daughterboards as a large portion is sensitive analog lines. The mainboard has more complex circuitry but it is “less interesting” as there is no external interaction with an ASIC.

On the surface the mainboard has the following tasks: regulating its own voltage, the STM32 corresponding circuitry, ADBMS communications via the ADBMS6822 chip and corresponding circuitry, current sensing, isolation relay control, and sending data over CAN.

Starting with voltage regulation, the vehicle has a power and ground line that runs throughout the harness. These come from the low voltage battery, made from the same cells as the accumulator, connected in a 6 series 3 parallel configuration resulting in a max voltage of 25.2V and a min of 15V. Due to changing in voltage and the chips on this board taking both 3.3V and 5V, we have to

regulate this voltage. Rather than regulating once at the battery and sending that regulated voltage throughout the car we choose to run the unregulated voltage because it is a higher voltage so we have less current flows and we get lower transmission line losses. Also most boards use either 3.3V or 5V (or both) and as a result twice as many wires would be needed in the harness. Regulation is done by switching converters. On this team we have standardized this component and use the Traco TSR switching converters. Specifically, the mainboard uses the TSR-1-2433 and the TSR-1-2450 regulating to 3.3V and 5V respectively. These come with built in inductors and capacitors, making them very easy to use. To better decouple the regulator, a 4.7uF bulk and a 100nF decoupling capacitor were added to V_in on each regulator.

In case the unregulated line was plugged in backwards, reverse polarity protection was added. This was done with a PMOS with drain tied to the unregulated input, source tied to the input to the regulators, and gate tied to the drain (Figure 20). The PMOS has a Vds of 30V but a Vgs of only 25V. Because the LV battery (unregulated line) can go up to 25.2V, a voltage divider to the gate was placed to lower the Vgs in the event of a reverse connection. Also like most signal lines on this board, a decoupling cap was placed from the gate to drain. A 0 Ohm DNP resistor was placed across the PMOS in the event that this circuitry was incorrect so the PMOS could be bypassed without needing to respin the board.

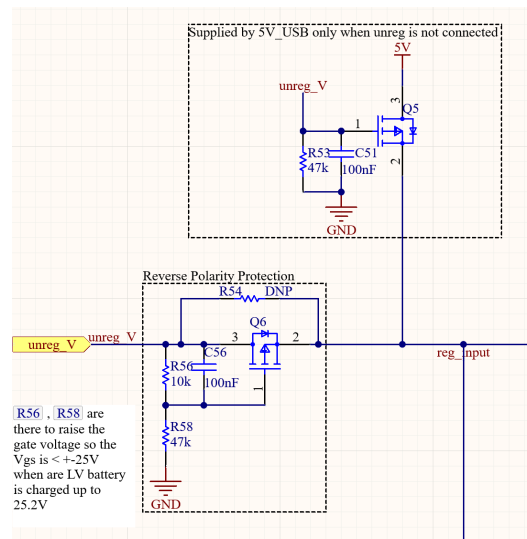


Figure 20: Reverse Polarity and 5V supply schematic

In order to not constantly need a power supply or battery to supply unregulated when testing, a similar PMOS was placed from 5V to the regulator input with the gate tied to the regulator input (Figure 20). The PMOS turns off when unregulated is present as to not short 5V to unregulated. However, when unregulated is not present, the PMOS turns on and allows 5V to supply voltage to the regulators. Normally a Schottky diode would be used, but as this line is already 5V so the voltage drop across the Schottky would be undesirable. 5V is high enough above the dropout for the 3.3V regulator such that it can still work, and the supplied 5V through the micro usb will directly power the chips needing 5V.

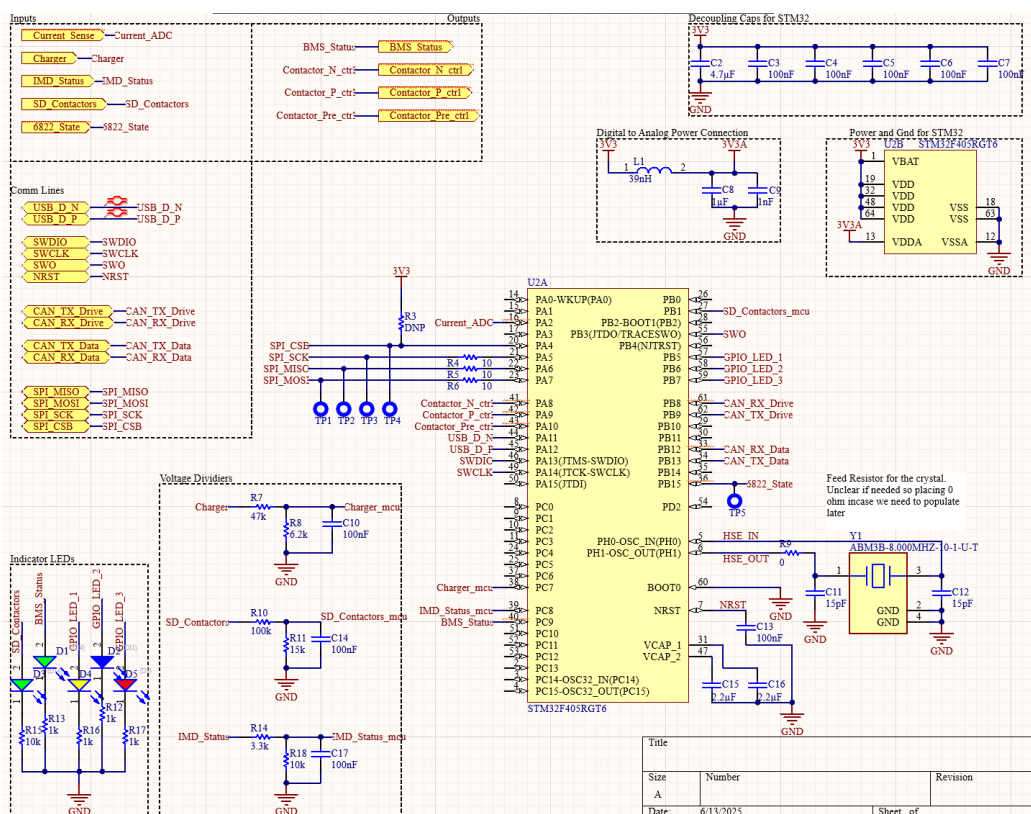


Figure 21: Schematic for STM32

The schematic for the STM32 corresponding circuitry is shown in Figure 21. Decoupling capacitors were placed where the SMT32 requires them, and on every power pin. An inductor was added to tie the 3.3V power to STM32 to the STM32 analog power. A quartz crystal was added to help reinforce the timing of the chip due to having both SPI and CAN communication. The 0 Ohm resistors in series with the crystal output is a feed resistor that is sometimes required

to limit the current flowing into the crystal. Lastly there are 3 voltage dividers. The charger, shutdown contactors and IMD status. These are needed so the SMT32 can read the digital status of these lines (high or low). The charger line is at 24V regulated, the shutdown contactors is the same voltage as the LV battery, and the IMD status is at 5V. The STM32 has a max GPIO voltage of 3.3V, resulting in the need to divide this voltage down to be in an appropriate range for the SMT32. The BMS reads the IMD status so it can send it to the rest of the car (namely ECU). It has to do this because IMD status is determined by the HV board and the HV board is strictly hardware.

Status LEDs were placed throughout the board to visibly show specific digital line values and are described in Figure 22.

<u>LED colors chart</u>	
Green	- 3.3V
Green	- Shutdown_Contactors
Green	- BMS_Status
Red	- External Fault
Yellow	- Startup
Yellow	- CSB
Blue	- Cell Balancing
Blue	- Contactors

Figure 22: LED color code chart

The BMS status out line that originates from the STM32 and gets set to other boards on the car needs to be at 5V. As a result, a 3.3V to 5V level shifter was added (Figure 23).

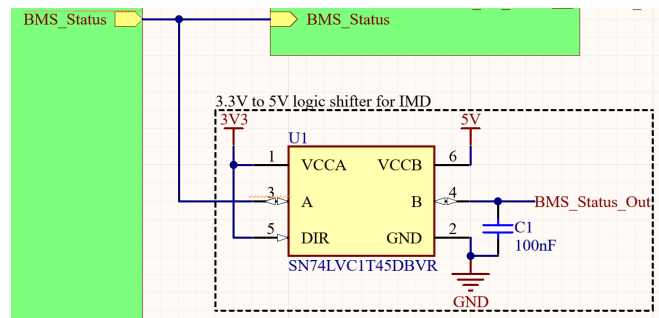


Figure 23: BMS Status 3.3V to 5V Level Shifter

Two CAN transceivers were added on this board so the BMS can talk on both of the CAN lines we have in the vehicle. The CAN transceivers are standardized across the team. Data can is the CAN line that the sensors and other data acquisition boards talk over. On this line, the BMS spits out every voltage and temperature that it measures. Drive can is the CAN line that drivetrain electronics talk over. This is where the BMS sends out the BMS status (and what fault is present, if any), IMD status, max cell voltage, min cell voltage, max cell temperature, min cell temperature, battery voltage, and measured current. This can be thought of as the important data that the rest of the car needs in order to drive. The drive CAN line is also where the BMS receives the ECU, Inverter, and Charger CAN messages. Very late into this year, we decided to forgo having two CAN buses and simplify it to just one. This does not change the fact that the hardware is already laid out and working boards are in hand, but it is worth noting that one bus is entirely unused and should be removed, if we continue to have two CAN buses going forward in future years.

The BMS controls 3 isolation relays (also called contactors or IRs) that control if tractive system (TS) voltage is present outside of the accumulator. There are two high current relays, one for TS+ and one for TS-. There is also a precharge relay. This relay along with a 6k Ohm resistor in series is connected across one of the high current relays to provide a current limited way to charge up the TS outside of the accumulator. This is necessary because the inverter has a large DC-link capacitor that has to be charged to the tractive system voltage before both high current relays are close to prevent a short. There is a circuit that runs throughout the vehicle called the shutdown circuit. This is a latching circuit i.e. if something is high it stays high until reset. The shutdown circuit is what directly powers the contactors. The BMS gets the shutdown circuit in and needs to switch it once based on if there is an internal BMS fault. The shutdown line then goes out of the BMS. The BMS then gets the shutdown circuit back at the end of the circuit and uses this to control the isolation relays. As a result, the BMS needs 4 switches to control the circuit. Once the first time and one switch per isolation relay.

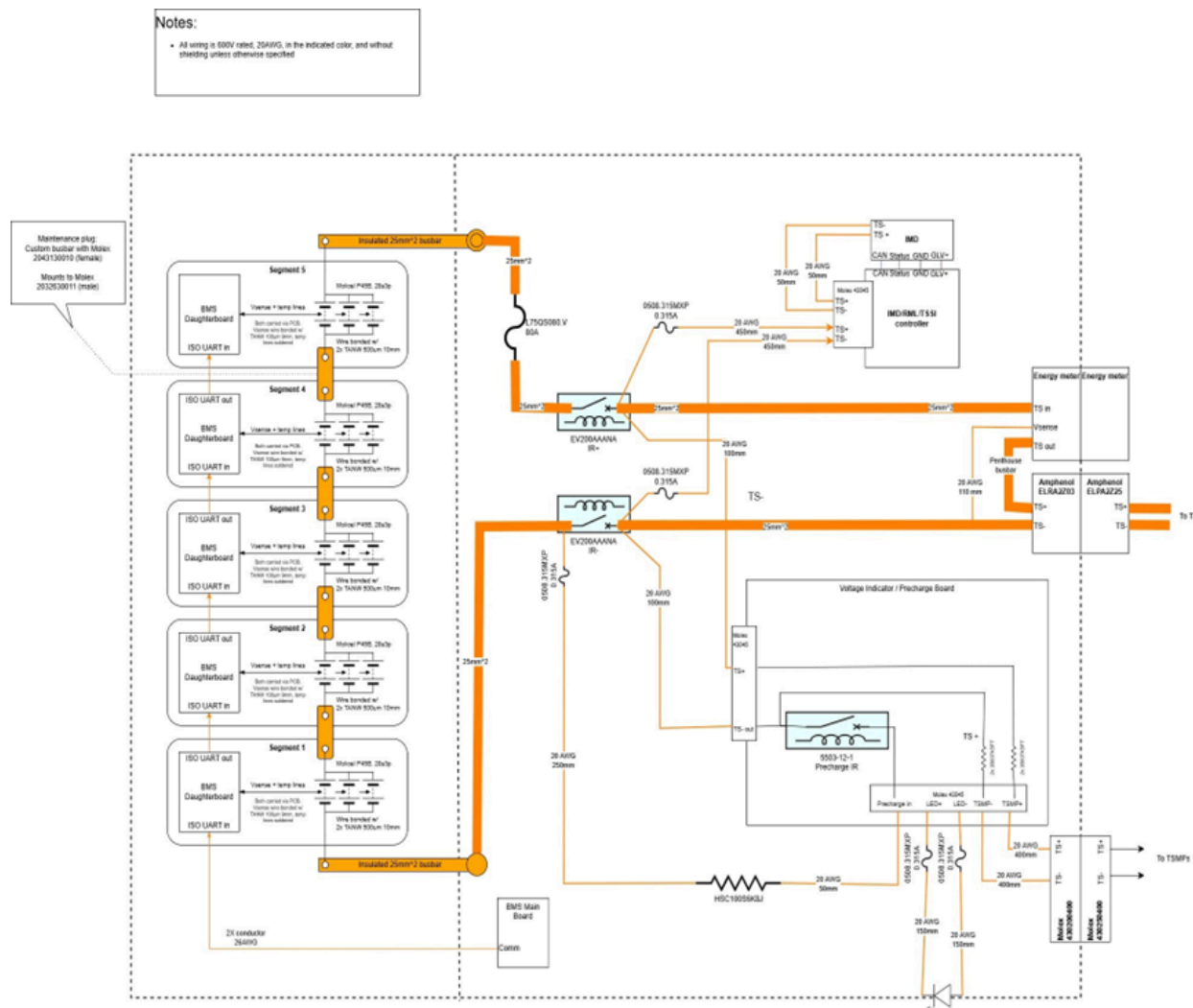


Figure 26: Accumulator TS Schematic

The switch is controlled by the STM32. The signal lines go into the gate of an NMOS that is used to invert the signal and drive a PMOS that is in series with the shutdown line. The gates of both FETs are pulled to the drains so that in a case with no signal from the SMT32 it defaults to off. Much like the reverse polarity protection PMOS, this PMOS has a voltage divider to drive the gate due to the V_{gs} of 25V and max shutdown voltage of 25.2V. Again like most signal lines, there are decoupling caps placed on the gates of the transistors. Lastly, there is a flyback diode placed across the isolation relays to limit a large voltage being generated by the inductor in the isolation relays. Figure 26 displays the schematic for one of the 4 shutdown switches. The rest are more or less identical.

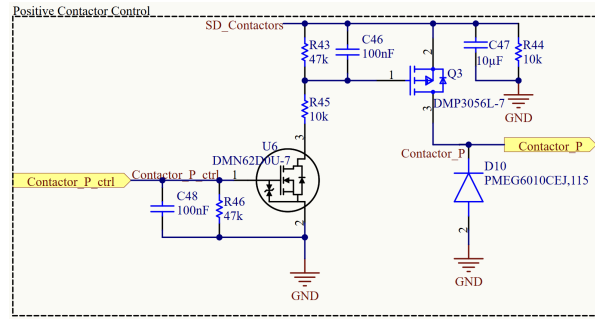


Figure 27: Isolation Relay Control Schematic

Finally the mainboard current sensing. This is arguably the most interesting aspect of the mainboard circuitry. At a high level, there is a 100uF shunt resistor connected in series with the high current path. Because of this a small voltage is developed across the resistor dependent on the current. The mainboard takes this small differential voltage, isolates and amplifies it and turns it into a single analog signal that the STM32 can read. It does this through an isolated amplifier that galvanically isolates the tractive system. Specifically the AMC3302DWE isolated amplifier that has a fixed gain of 41. This chip has an integrated isolated power supply and has dedicated pins to the connection across the shunt resistor to maintain the kelvin connection. The isolated amplifier spits out an isolated and amplified differential pair. The differential signal is then fed into a differential amplifier which is low pass filtered and fed into an ADC on the STM32. The differential amplifier is offset with a reference voltage of 1.25V according to the schematic in Figure 28. The purpose of this is because current can flow in both directions and such the measured voltage could be either positive or negative and so there needs to be an offset to read current in both directions.

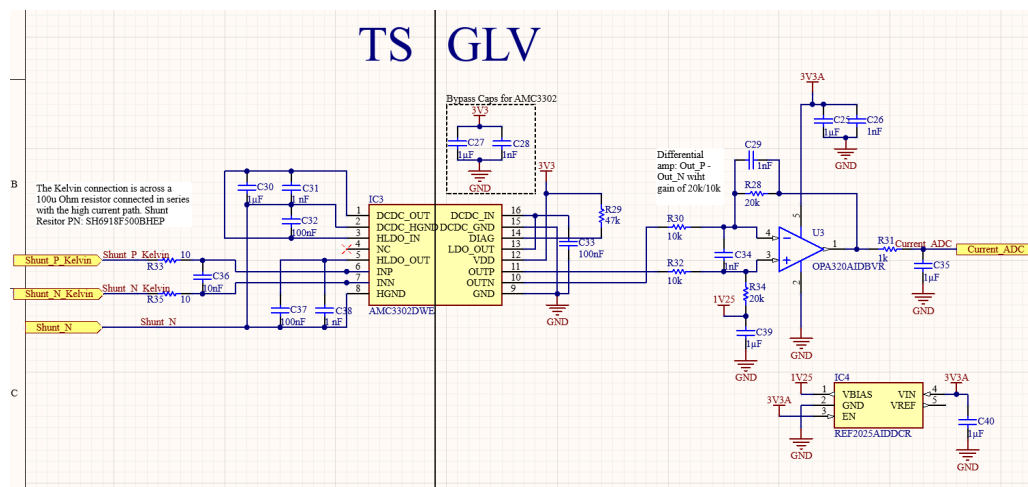


Figure 28: Current Sensing Schematic

The shunt resistor has a resistance of 100u Ohms. For the max continuous current of 135A this results in 13.5mV * Fixed gain of 41 = 0.55V differential output of the isolated amp. Budgeting for worst case +-200A, the isolated amp output voltage is 0.82V. In order to maximize the precision, a gain was chosen to use the full ADC range of 0 to 3.3V over +-200A. Having a reference of 1.65V and a gain of 2 results in a max voltage of 3.29V at +200A and a min of 0.01V at -200A. The STM32 ADCs are 12 bits resulting in ~0.1A/LSB.

3.2.A Layout

The layout is not as notable as the daughterboard. Care was taken to minimize the area while still maintaining signal integrity, including separating the analog current sensing circuitry from the rest of the circuitry via star grounding. Other best practices were followed such as placing decoupling capacitors as close to the pins as possible.

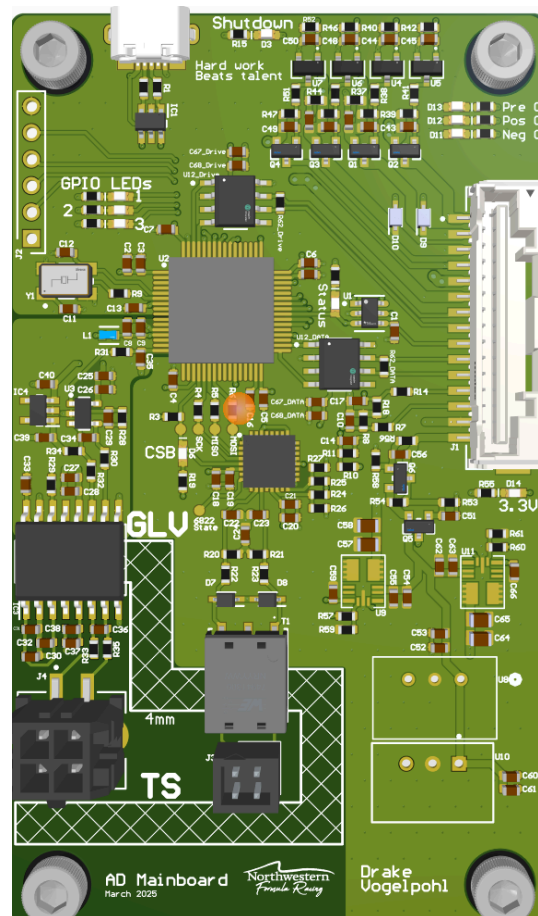


Figure 29: 3D view of mainboard layout

The only non-self explanatory aspect of the layout is the TS keepout zone. Because this board has both tractive system and grounded low voltage (every electrical part not part of the tractive system) present, a keepout zone is required due to the high voltage differential and by FSAE rules shown in Figure 30.

EV.6.5.7 If Tractive System and GLV are on the same circuit board:

- a. They must be on separate, clearly defined and clearly marked areas of the board
- b. Required spacing related to the spacing between traces / board areas are as follows:

Voltage	Over Surface	Thru Air (cut in board)	Under Conformal Coating
0-50 V DC	1.6 mm	1.6 mm	1 mm
50-150 V DC	6.4 mm	3.2 mm	2 mm
150-300 V DC	9.5 mm	6.4 mm	3 mm
300-600 V DC	12.7 mm	9.5 mm	4 mm

Figure 30: FSAE Rules for TS and GLV on the same board

The hatched stripe is 4mm wide and denotes where the keepout zone is that separates the tractive system from the rest of the board with only the galvanic isolation crossing over the keepout.

3.2.B List of improvements

- Due to the high voltage present in the accumulator any small measurement error in current results in large error in charged/discharged power. MCUs like the STM32 do not have very high precision ADCs because it is not necessary, and if high precision is needed external ADCs are used. To solve this using the ADBMS2950 pack monitoring chip with highly sophisticated ADCs designed for current measuring could be used.
- The briefly mentioned HV board that does IMD latching, Ready to Move Light, and Tractive System Indicator Lights lives next to this board in the penthouse. Now that we have a better understanding of the electronics for the BMS and other rules required items it would make sense to combine these boards into one.

4 Firmware

The firmware is broken into two sections. The “low level” section interacts with the ADBMS chips to get the voltages, temperatures, and status register values and perform open wire checks and cell balance when appropriate. The “high level” section focuses on the functionality of using the custom BMS as the brains of the accumulator and parsing through the data collected by the low level section.

4.1 AD BMS (Low Level)

The low level code is mainly interacting with ADBM6830B chips that live on the daughterboards to do the following actions: configure the chips, get the status registers, get raw voltage values, get raw temperature values, perform open wire check, and cell balance when necessary. It does so via drivers we wrote and an interface layer that calls these drivers to get raw values then process them to usable values. These processed values are then used to update internal faults in the AD internal faults step. That is where the low level stops. Every variable the low level updates are called AD variables/values (any variable or value that is updated with data from ADBMS630B chips). For complete understanding, the high level section calls the low level functions (update, calculate and process faults), reads current, reads external values, and updates the finite state machine context in the update values steps. The high level then evaluates the finite state machine and sends/receives CAN messages with the values updated in the previous step.

To reiterate the low level steps are: configure the daughterboard chips on startup, get raw AD values, process the AD values, and update internal faults. These steps are done in the two files `adbms_driver.c/h` and `adbm_interface.c/h` where the interface file makes use of the driver file to get the raw AD values. This section will be going over these two files and explaining some of the less clear decisions that were made and problems and the solutions to those problems that came up when trying to communicate with daughterboards.

To start, the chips are configured with the following settings: turning on the reference voltage, turning every internal GPIO pull down off so they can be used for temperature sensing, setting

the internal overvoltage and undervoltage in the hardware of the ADBMS6830B chips, and turning off every cell internal balancing switch (Figure 31). After setting the chip configurations, cell sensing on the c channel is turned on with continuous measurement. This means the c channels will update their values continuously until turned off or the chips go into sleep mode. Sleep mode is whenever the chips go 2 seconds without being communicated with and the reason for the wakeup command. The c channel voltage sensing is set with the redundant measurements turned on, discharge off, and open wire off. Next the auxiliary channels, the channels temperature sensing is measured on, are turned on. This is done with auxiliary open wire switch off, pull up turned off, and the channels muxed to ADC functionality. As a side note, the auxiliary channels also support other behaviors than just analog to digital and have internal pull ups/downs. The Aux channels also do not support continuous polling in the same way that the C and S channels do. Because of this after each read, the Aux channels measurement command needs to be re-sent.

```
// Set initial configurations
for (uint8_t cic = 0; cic < NUM_CHIPS; cic++)
{
    // Init config A
    adbms->cfa[cic].refon = 1;
    adbms->cfa[cic].gpo = 0x3FF; // all gpo turned on

    // Init config B
    adbms->cfb[cic].vuv = Set_UnderOver_Voltage_Threshold(UNDERVOLTAGE);
    adbms->cfb[cic].vov = Set_UnderOver_Voltage_Threshold(OVERVOLTAGE);
}

// Init sensing cmd
adbms->adc.v.cont = 1;

// Package config and sensing structs into transmittable data
ADBMS_Set_Config_A(adbm->cfa, adbm->ICs.cfg_a);
ADBMS_Set_Config_B(adbm->cfb, adbm->ICs.cfg_b);
ADBMS_Set_ADCV(adbm->adc.v, &adbm->ICs.adcv);
ADBMS_Set_ADAX(adbm->adax, &adbm->ICs.adax);

// Write Config
ADBMS_WakeUP_ICs_Polling();
ADBMS_WakeUP_ICs_Polling();
ADBMS_Write_Data_RegGrp_Polling(adbm->ICs.hsapi, WRCFG_A, adbm->ICs.cfg_a, adbm->ICs.spi_tx_dataBuf);
ADBMS_WakeUP_ICs_Polling();
ADBMS_Write_Data_RegGrp_Polling(adbm->ICs.hsapi, WRCFG_B, adbm->ICs.cfg_b, adbm->ICs.spi_tx_dataBuf);

// Turn on sensing
ADBMS_Write_CMD_Polling(adbm->ICs.hsapi, adbm->ICs.adcv);
HAL_Delay(1);
ADBMS_Write_CMD_Polling(adbm->ICs.hsapi, adbm->ICs.adax);
HAL_Delay(8); // ADCs are updated at their conversion rate of 1ms
```

Figure 31: AD chip configuration

The ADBMS_Set_Config_A, ADBMS_Set_Config_B, ADBMS_Set_ADCV, ADBMS_Set_ADSV, ADBMS_Set_ADAX are functions part of the driver library used to package the cfa_, cfb_, adcv_, advs_, and adax_ structs into a buffer that is transmittable to the 6830 chips. While the structs and the functions themselves require a decent amount of bit shifting to package correctly, they are very straightforward and directly follow the datasheet. As a result, it is not worth discussing them in detail here. ICs is an array of type adbms6830_ICs, again defined in the library that holds the spi handler, rx/tx buffers as well as the buffers the raw data gets placed into.

After setting the correct chip configuration the AD values are needed to be updated every loop. This is done by the ADBMS_UpdateVoltages and ADBMS_UpdateTemps functions. These make calls to the driver to get the raw data, and then process that data.

```
void ADBMS_UpdateVoltages(adbms_ *adbms)
{
    // get voltages from ADBMS
    uint8_t pec = 0;
    ADBMS_WakeUP_ICs_Polling();

    pec += ADBMS_Read_Data_RegGrp_Polling(adbms->ICs.hsapi, RDCVA, (adbms->ICs.cell + 0 * NUM_CHIPS * DATA_LEN), adbms->ICs.spi_tx_dataBuf);
    pec += ADBMS_Read_Data_RegGrp_Polling(adbms->ICs.hsapi, RDCVB, (adbms->ICs.cell + 1 * NUM_CHIPS * DATA_LEN), adbms->ICs.spi_tx_dataBuf);
    pec += ADBMS_Read_Data_RegGrp_Polling(adbms->ICs.hsapi, RDCVC, (adbms->ICs.cell + 2 * NUM_CHIPS * DATA_LEN), adbms->ICs.spi_tx_dataBuf);
    pec += ADBMS_Read_Data_RegGrp_Polling(adbms->ICs.hsapi, RDCVD, (adbms->ICs.cell + 3 * NUM_CHIPS * DATA_LEN), adbms->ICs.spi_tx_dataBuf);
    pec += ADBMS_Read_Data_RegGrp_Polling(adbms->ICs.hsapi, RDCVE, (adbms->ICs.cell + 4 * NUM_CHIPS * DATA_LEN), adbms->ICs.spi_tx_dataBuf);
    adbms->total_pec_failures += pec;
    adbms->voltage_pec_failure = pec;

    // calculate new values with the updated raw ones
    ADBMS_CalculateValues_Voltages(adbms);
}
```

Figure 32: ADBMS_UpdateVoltages function

The voltage and temps update are very similar with the only difference is temps have to start a new aux conversion after every read, so only the update voltages will be discussed in detail. This makes heavy use of the ADBMS_Read_Data_RegGrp_Polling function from the drivers. This function simply takes a command, the spi handler, a tx buffer, and the array to place the raw data. It then packages the command into 4 bytes transmittable over SPI to the chips, including a 2 byte packet error check (PEC). It then makes use of the HAL_SPI_TransmitReceive to send and receive the raw data over SPI. Finally, it processes the raw data, determining if there was a PEC failure and packages the raw data into the provided array. The processing is shown in Figure 33.

```

bool ADBMS_Process_Read_Data_RegGrp(uint8_t *rx_dataBuf_raw, uint8_t *dataBuf)
{
    // Move Rx ptr to skip over data during transmit phase
    uint8_t *rx_dataBuf = rx_dataBuf_raw + CMD_LEN + PEC_LEN;

    bool pec_error = 0;
    for(uint8_t cic = 0; cic < NUM_CHIPS; cic++)
    {
        for(uint8_t cbyte = 0; cbyte < DATA_LEN; cbyte++)
        {
            dataBuf[cic * DATA_LEN + cbyte] = rx_dataBuf[cbyte + (DATA_LEN+PEC_LEN)*cic];
        }
        uint16_t rx_pec = (uint16_t)((rx_dataBuf[DATA_LEN + (DATA_LEN+PEC_LEN)*cic] & 0x03) << 8) | rx_dataBuf[DATA_LEN + 1 + (DATA_LEN+PEC_LEN)*cic];
        uint16_t calc_pec = (uint16_t)Pec10_calc(true, DATA_LEN, (rx_dataBuf + cic * (DATA_LEN + PEC_LEN))); // Needs the PEC to calculate the PEC
        pec_error |= (rx_pec != calc_pec);
    }
    return pec_error;
}

```

Figure 33: Driver Processing from Read Data

The commands are used to tell the chip what you expect it to be doing, and are described in length in the datasheet, but here the commands are simply getting a register group. Here it is getting the C-channel register groups.

Once the AD data is updated in the ICs array from the drivers, it needs to be processed from raw values to voltages. Raw values of type int received from the daughterboards are converted to a voltage of type float according to the following mapping that comes from the 16-bit ADCs having a LSB of 150 uV.

$$Voltage = (raw\ value + 10000) * 0.000150$$

In the calculate values step (done by the ADBMS_CalculateValues_Voltages function) these raw values are turned to voltages and are placed in the high level struct along with the max, min and average values. The voltages are done according to the equation above.

The temperatures are a little more complicated and are done in the ADBMS_CalculateValues_Temps function. The aux measures the voltage across the thermistors. Knowing the voltage across the thermistor, the voltage divider reference voltage, and the value of the other resistor of 10k Ohms, the resistance of the thermistor can be calculated according to:

$$R_{thermistor} = \frac{V_{thermistor} R_{series}}{V_{ref} - V_{thermistor}}$$

This resistance can then be mapped to temperature according to the thermistor datasheet. In our case, the thermistors are NTC thermistors with a B value of 3435, T_0 of 298.15 kelvin, and a

resistance of 10k Ohms at T_0 and can be mapped according to the standard NTC thermistor equation (Figure 34).

$$T = \frac{B}{\ln \left(\frac{R_{thermistor}}{R_0 \times e^{\frac{-B}{T_0}}} \right)}$$

Figure 34: NTC Resistance to Temperature Equation

Conveniently, open wire detection for the thermistors can be done here. Open wire detection will be described in more detail later, but is a check to see if a signal to the board is disconnected via fusing or otherwise. In this case if a thermistor is disconnected there is no voltage divider, the 10k Ohm on-board resistor just becomes a pull-up, pulling the ADC to the reference voltage. Exploiting this, if the measured temperature voltage equals the reference voltage (within a threshold of 100mV to be safe) the temperature open wire flag can be set.

When talking to ADBMS6830B chips, if the connection is broken (e.g. disconnecting the comms line) or the chip is otherwise not responding, the drivers will not hang or otherwise show any indication that it is not making a connection. What will happen is it will read all 0s and the PECs will fail. This is reported by the driver read functions returning 1 (instead of 0 if everything is correct) and handled at the interface level by tracking the number of failed PECs and if it is greater than a threshold raising a fault. The purpose of this is that packets might get dropped during long/intense periods of use. That being said, if a singular packet is dropped, that is not a problem as it will just get the voltage the next time around (<0.2 seconds) and is not a significant issue. However, if many packets in a row are being dropped (due to a disconnected comms line or otherwise) and thus the voltages are not being updated the PEC fault will be raised.

After updating and calculating the AD voltage and temperature values internal faults need to be set. The internal faults are overvoltage fault, undervoltage fault, over/under temperature fault, open wire fault, and PEC fault. Note that the temperature open wire and PEC faults are already handled as described above.

As described above, open wire check from a software perspective should be very straightforward: close the even OWC switches and set a fault if a 0 is read, then open them and close the odd OWC switches and set a fault if a 0 is read, then open them. The chip provides 1 bit in memory that can be set to close the odd switches and one bit for the even switches. The drivers mimic this functionality by setting the OWC bits in `adsv_` struct. It is also worth noting that OWC does not have to be checked as frequently as updating the rest of the AD values. The voltages and temperatures are constantly needed to be measured for the safety and state of charge estimation of the pack. The open wire detection does not need to be updated as frequently because if there is an open wire the car will immediately turn off. Also an open wire is not as dynamic as the voltages and temperatures and thus requires a lower frequency. Where a lower frequency might mean once every 10 seconds (if not longer).

The last function of the low level code is cell balancing. Unlike the rest of the low level functionality, this only happens when charging and thus will never be turned on while driving. Due to manufacturing tolerances and chemistries in cells, groups of series cells will naturally unbalance when charging or discharging. To fix this they need to be balanced hence the name cell balancing. This can be done in one of two ways, active and passive. Active is when energy from higher charged cells are used to charge lower cells i.e. redistributing charge from higher charged cells to lower ones. Passive is done by dissipating charge from higher charged cells to match the charge of lower level cells through a resistor. This effectively bleeds the charge away as heat. While active balancing has obvious benefits, in our case, the complexity far outweighs the benefits. Especially due to the fact that the cells we use have a fairly tight tolerance and thus will not become very unbalanced after just one discharge. Balancing is only done when charging because we do passive balancing. As passive balancing essentially wastes the energy as heat it is harmful to do when we are trying to squeeze the most out of our pack while racing. On a similar note, balancing is necessary to do while charging because charging has to conclude when the first cell reaches its max voltage. This means if the cells are not balanced we are leaving potential charge on the table. As shown in Figure 35, on the s channels are dissipation resistors and an internal PMOS that can be closed to enable cell balancing.

Cell balancing can be enabled in two ways on the ADBMS6830B chips: continuous or PWM. Exactly as it sounds continuous, continuously closes the PMOS while PWM will close the PMOS according to the PWM duty cycle. For our use case, continuous works perfectly fine and is much simpler to enable, simply writing the specific bit high will turn on the switch. This chip also supports discharging in sleep mode when in PWM with a discharge timeout set. While this is a super cool feature, it is quite complex to implement, as we only balance when charging this would not be useful. It also could potentially get us in trouble by draining the pack if enabled incorrectly. There are more harms to balancing in sleep mode (extended balancing state in the datasheet) but not worth getting into here. While charging, balancing will be enabled and every cell above a threshold above the minimum cell will be turned on (Figure 36), and it is then turned off after charging is concluded or when open wire check is performed due to reasons mentioned above. Note the dcc is a 16-bit int that corresponds to the 16 different s channel cell balancing switches. To turn on a specific channel that bit is set to 1, thus to turn off cell balancing dcc is set to 0.

```
void cellBalanceOn(adbsms_ *adbsms)
{
    // Turn on CB indication LED
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);

    for (int cic = 0; cic < NUM_CHIPS; cic++)
    {
        uint16_t dcc = 0;
        for (int cvoltage = 0; cvoltage < NUM_VOLTAGES_CHIP; cvoltage++)
        {
            float curr_v = adbsms->voltages[cic * NUM_VOLTAGES_CHIP + cvoltage];
            if ((curr_v - adbsms->min_v) > CB_THRESHOLD && curr_v > CB_MIN_V_THRESHOLD)
            {
                dcc |= 1 << cvoltage;
            }
        }
        adbsms->cfb[cic].dcc = dcc;
    }

    ADBMS_Set_Config_B(adbsms->cfb, adbsms->ICs.cfg_b);
    ADBMS_Write_Data_RegGrp_Polling(adbsms->ICs.hsapi, WRCFGB, adbsms->ICs.cfg_b, adbsms->ICs.spi_tx_dataBuf);
}
```

Figure 36: Cell Balancing Function

4.1.A List of Improvements

- S channels can be used to perform a voltage measurement when not cell balancing or performing open wire check. While these channels provide more noisy values, they can be compared against their c channel counterparts as a redundant measurement. The datasheet further describes this.

- This is not mentioned, but there is a configurable internal IIR or FIR hardware filter present on the c channels. It is worth looking into how to enable this and validating if filtering provides more accurate values.
- In a similar notion as the hardware filter present on the ADBMS6830B chips, it would be worth looking into having a software filter for the AD values.

4.2 NFR BMS (High Level)

The high level code focuses on using the AD values collected by the low level section and collecting its own values (both through CAN and GPIOs) to act as the brains of the accumulator including communication with the rest of the car, defining the accumulator states, and controlling the isolation relays.

Before diving into how the high level section functions, it is worth mentioning one thing. This section of the code should not exist. At least at the same complexity it currently does. This point will be elaborated on in section 4.2.C.

Instead of describing the direct implementation as the previous section, this will go over the logic present and a verbal description of how it was implemented. This is because the logic is what is important here and the implementation directly follows from the logic.

The functionality of the BMS can be broken down into initialization and 2 timer groups. A timer group is just a way of calling a function only after the designated time has passed based on the internal clock of the STM32. The reason for doing this is that without a timer group the STM would continually process these loops quicker than is required and having a timer group allows functions to be called at different rates. The 2 timer groups are the mainboard loop and OWC loop.

Initialization purely consists of hardware initializations of the GPIOs, ADCs, SPI, and CAN buses and setting the configurations of the daughterboards via writing to the ADBMS6830B chips as described in section 4.1.

The mainboard loop can be thought of as the loop that performs the functionality of the BMS. It calls the low level functions, reads current, reads external values, and updates the finite state machine context in the update values steps (Figure 37). It then evaluates faults in the check faults step. Finally, it then evaluates the finite state machine.

```
void UpdateValues()
{
    // ADBMS values
    ADBMS_UpdateVoltages(&mainboard.adbms);
    ADBMS_UpdateTemps(&mainboard.adbms);
    UpdateADInternalFault(&mainboard.adbms);

    // update STM32 Pin values
    // reads: shutdown_contactors, IMD_Status, 6822_State
    mainboard.shutdown_present = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_1); // shutdown status
    mainboard.imd_status = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_8); // IMD_Status
    mainboard.comms_6822_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_15); // 6822_State
    mainboard.charger_pin = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_7); // Charger_Pin

    // get current
    mainboard.current = getCurrent(mainboard.hadc) - mainboard.current_offset;
    mainboard.overcurrent_fault = mainboard.current > OVERCURRENT;

    soc_update(&mainboard);

    if(ENABLE_PRINTF_DEBUG_COMMS) send_data_over_printf();
    if(ENABLE_USB_COMMS) send_data_over_USB();
}
```

Figure 37: Update Values function

The current is gathered by reading the ADC voltage, and mapping that to current based on the hardware described in section 3.2. It takes the raw 12-bit value read from the ADC and maps it to voltage by dividing by 2^{12} and multiplying by 3.3V as it is a 3.3V ADC. That voltage is then subtracted by the reference voltage and mapped to current by dividing by the gain and shunt resistance.

The state of charge (SOC) is done by naïve Coulomb counting. On startup the open circuit voltage of the cells is mapped to an initial SOC or $SoC(t=0)$. After startup, the SOC is tracked by taking the measured current and the time since the last SOC update to satisfy the following equation. Where Q_n is the total ampacity of the system. For us, we have 3 4.5Ah cells in parallel so the total ampacity is 13.5Ah.

$$SoC(t) = SoC(t - 1) + \frac{I(t)}{Q_n} \Delta t$$

Figure 38: Coulomb Counting Equation

```

void adbms_mainboard_loop()
{
    UpdateValues();
    CheckFaults();
    fsm_run(mainboard.fsm);
    drive_can_loop();
}

```

Figure 39: Mainboard Loop

The check faults functions check for both internal and external faults. An internal fault is defined as a BMS fault (overvoltage, undervoltage, over temperature, open wire, and overcurrent) and has to be latching (i.e. if ever pulled high, it has to stay high) due to FSAE rules. The internal faults are what drive the BMS state, where no faults present have a healthy state of 1 and if there are any faults the state is 0. External faults are any external condition that would stop the BMS from closing the isolation relays and are no shutdown circuit present (the line that drives the relays) or a timeout from either the ECU (electric control unit). A timeout is defined as going over a set time without receiving a CAN message from a particular controller. This is quite easy to check for as the time of the most recent message can be stored and if the time between then and the current time is ever greater than a threshold, a timeout is set. The idea is if any of these controllers stop communicating the isolation relays should be open for safety.

The BMS also sends all of its data over CAN messages. It does so with 3 “overview” can messages that encompass the important variables that are needed to properly inform the rest of the car of the state of the accumulator. This includes: battery voltage, battery temperature, battery current, every fault status (both internal and external), the BMS state, the IMD state, max cell temperature, min cell temperature, max cell voltage, min cell voltage, and estimated battery state of charge. This is what the ECU and dashboard listen for. The BMS also sends out every single measured cell voltage and temperature in 30 other messages for the purpose of logging. The

Evaluating the finite state machine (FSM) is remarkably straightforward due to the lightweight FSM library by Evan Bertis-Sample. Instead of evaluating the stats of the FSM through a case-switch this library breaks it down into: on enters called when entering a state, on update call when staying at a state, on exits called when exiting a state, and predicate functions that dictate when to move between states.

the accumulator and the DC-link cap to be charged. Precharge is switched to Active once the DC-link cap is fully charged designated by the inverter voltage. Because there are equivalent series resistances between TS+ and TS-, there is a voltage divider between the 6k Ohm precharge resistor and the voltage present across the DC-link cap. For this reason the system cannot precharge up to the full voltage of the accumulator. Knowing this, FSAE rules dictate precharging only has to occur until the voltage is 90% of the accumulator voltage. So precharge is switched to active when the inverter dc voltage is 90% of the accumulator voltage. Active means closing the negative relay, no longer current limiting the tractive system. Precharge and active can be switched to the idle state if ECU ever says to open the contactors or an external fault is present.

The charging states operate on the same principles, but based on feedback from the charger rather than the ecu and inverter. Going into charging is done by pulling the charging pin high. In the vehicle harness this pin is left floating (pulled low on the mainboard via the voltage divider), but in the charger harness this pin is pulled high. When pulled high, the BMS starts sending a specific CAN message that the charger reads. This message contains the max charging voltage, max allowable charging current, and whether to enable charging or not. The max allowable charging voltage and current are system parameters based on the battery that is being charged and the mains supply being used to power it (as to not blow a mains fuse). The enable charging bit is a 1 to enable charging and a 0 to disable. In charger setup this is set to 0 to disable charging. In nominal operation, the charger sends a CAN message with the voltage present at its terminals, current, and fault status. The fault status will be a 1 (fault detected) until it receives a message from the BMS. Once that fault is cleared, charger setup can be moved into charger precharge (positive and precharge relay closed). Although there is no DC-link cap in the charger, the idea is to limit the tractive system current until the charger completes the handshake by responding with a voltage consistent with the measured tractive system voltage. Once this is checked, charger precharge can be moved to charging. This closes the negative relay removing the current limit and sets the charging bit to 1 to enable charging. If the charger ever stops responding or responds with a fault the state will move back into charger setup. On update while charging, cell balancing is turned on with the described functionality in section 4.1, and cell balancing is turned off when exiting the charging state.

Every state has a potential transition to the fault state if an internal fault is ever detected. Once in the fault state, it is not possible to exit without power cycling, even if the fault is cleared implementing the latching functionality required by the FSAE rules.

4.2. DMA and RTOS

Using the timer libraries works fine, but it is quite sloppy as there can be blocking functions and things could operate at even more specific times to squeeze better performance out of the BMS. In order to achieve this, it would make sense to run off of a Real Time Operating System (RTOS), FreeRTOS. This allows for specific tasks to be scheduled and happen at specific times.

A large benefit of the RTOS would be materialized if updating the AD values was done through interrupt or direct memory access (DMA). The reason for this is the majority of time is simply spent waiting for the response from the polling SPI. It would be ideal to have a constant stream of data from the chips in the background and evaluate everything else scheduled via an RTOS.

4.2.B State of Charge Estimation

There are very interesting algorithms to estimate the state of charge (SOC). To keep a long story short there is no good way to measure the SOC because the voltage of the cells drop under load (and have hysteresis). The driver still needs to know how much of the battery is left to inform how hard to drive, so the SOC needs to be estimated. To start, a rudimentary Coulomb Counting method was implemented to track the current discharged from/charged into cells while driving to estimate the amount of charge left. There are a host of issues with this algorithm, and very interesting algorithms that are proposed to get more accurate results. For more information, look at this presentation on [state of charge estimation algorithms](#).

4.2.C Ideal NRF BMS Behaviour

Northwestern Formula Racing has imposed this external functionality on our BMS to act as the “brains of the accumulator”. This is very unnecessary and actively harmful. There is already a “brain for the vehicle” being the ECU. Having the BMS and ECU both act as brains means their interactions have to be handshakes causing complicated interactions and confusion. Having this

functionality imposed on the BMS also means that its main goal of measuring voltages and temperatures gets overshadowed by the FSM behavior and interactions with the rest of the car.

In an ideal world there would only be one brain, the ECU, and the BMS can serve purely as a slave. Simply getting the AD values, calculating faults and SOC, and sending important information to the ECU. In return, the ECU can deal with all the different states present in the car and simply tell the BMS what contactors to close at what times over CAN. This simplifies the BMS firmware as almost all of the high level code would go away, and this high level, low level split wouldn't need to exist. This would free up resources on the BMS and allow it to entirely focus on being simply a BMS. The best part is this wouldn't change the hardware at all, and adding the BMS states to the ECU is negligible complexity. Unfortunately it is too late in the design process to make this change and this year's AD BMS had to be a drop in replacement for the BQ BMS resulting in this functionality. In future years I hope that this functionality is separated from the BMS.

5 Testing and Validation

This section covers the bench validation and then real world testing. It also covers a list of issues observed during this and how they were addressed or how it makes so go about addressing them in future years.

5.1 Using the Dev Boards

When we received the sponsorship from Analog Devices for this chipset they gave us the dev boards for the ADBMS6830B and ADBMS6822. The ADBMS6822 was designed in such a way that it can plug into the top of a Nucleo. In theory this is very beneficial for debugging the designed mainboard and daughterboards as their functionality can be compared to the dev boards. It also is helpful for writing software as there is professionally designed hardware that can be used to validate the functionality of certain firmware. However, the ADBMS6822 and the Nucleo were not making a proper connection. It is unclear if the issue is with the Nucleo or the ADBMS6822 dev board, but they would communicate properly only when pressure was put on them or they were flipped upside down. This was very difficult to debug as the dev boards working and our code having the error was assumed until painfully proven otherwise after hooking the dev boards up to a logic analyzer. Fortunately, the daughterboard and mainboard were designed correctly on the first try and they were able to be used to test and validate the firmware as it was being written.

5.2 Virtual Comm Port

This is a short section, but worth noting down. Because it was getting really obnoxious to constantly run the STM32 in debug mode and record the SVM terminal, we turned on the virtual comm port on the STM and had it print out its values over that. This simplified testing massively, as the USB port that provided power could now also be used to record data. The other benefit is having the virtual comm port write to the console allowed us to use programs that would auto write it to a text file. This had the added benefit of being very easily parsable in a Jupyter notebook to plot voltages and temperatures over time.

5.3 Custom Drivers

At a high level, the driver files are just reading and writing to specific addresses in memory of the ADBMS6830B chips. When we got the sponsorship for the ADBMS from Analog Devices, they provided us with their custom drivers. On top of that, AD uses the HAL SPI driver from STM32, so the only thing their files are doing is designating which registers/addresses to write to and read from (same as the drivers we wrote). The addresses are described in the datasheet, and is no different from writing any other driver for a peripheral other than just the sheer amount being accessed, properly setting the PEC, and making sure there are no timing violations. Something we observed at the beginning of testing was using their drivers as intended, resulted in data being acquired way too slow for our system needs, and significantly slower than what the hardware supports (on the order of 2 seconds to get 10 chips worth of voltage and temp data). We also at the beginning of testing on the bench ran into an issue with needing a global variable for their ICs array, needing to add weird delays and config writes when doing OWC, and cell balancing not working on >4 chips in the stack (see winter quarter report). There are also countless other small interactions with 6830 chips that we observed that seemed to only exist when using their drivers.

Digging into their drivers a little bit revealed, to be quite blunt, that they are very convoluted, hard to read, and very inefficient. Because of this, I dove into their drivers and made a decent number of changes to get them working faster. I could go on a rant about these changes, but talking about them is not very productive for a couple of reasons. One, these changes were mostly patchwork and didn't address the larger issues of just being written in a very intuitive way. Two, their drivers are not publically available, and three, I wrote my own drivers (see section 4.1 for more details) that address the larger issues of their drivers that I open-sourced along with this entire project.

The countless issues we encountered occurred when we felt like we were operating the chips in a completely normal way, and the fixes we had to do were quite silly (e.g. adding delays). For this reason we thought an issue might just be with their drivers. This and the patchwork that we did on their drivers seemed to solve a lot of our issues was the motivation for writing our own drivers. Also note that in this report I omitted a lot of details from the custom drivers. To be

honest it is nothing too interesting other than following the datasheet. To see specific details of this just look at the repo and read the source code.

5.4 Cell Balancing

This is another issue that seemed to be caused by the drivers. To simplify multiple days of testing, cell balancing was working on the bench, but not on multiple segments. This was observed by bypassing the cell balancing algorithm and telling the chips to balance the bottom half of their cells and tracking every voltage over time. If cell balancing was working we would see that those bottom cells with cell balancing enabled would drop at a rate of roughly 4 times the normal cells. This is due to the static draw of 12 mA from the chip compared to the nominal cell balancing current across the cell balancing resistor of 36 mA, so a cell with cell balancing enabled would see 48 mA or 4 times faster voltage drop nominal.

Basically the issue was as described where for one board (two chips) cell balancing would be working but not when we moved to the full pack of 10 chips. For a number of reasons, mainly more pressing issues and all the cells being close enough ($<20\text{mV}$) to not warrant cell balancing put this issue on the back burner for a little bit. We had already switched over to using our custom drivers when we eventually got back around to testing this, and the issue was no longer present with our drivers. I don't really have a good intuition for why this would occur, but I would suspect it is the same reason as to why we had to add the random delays in OWC when using their drivers. Either way running the hard coded bottom half of the cells resulted in the following graph where the bottom 7 cells on each chip had a significantly steeper slope than the non-cell balancing cells (Figure 41). This is also backed by looking at the slopes of each cell voltage over the course of the test (Figure 42). Each dot corresponds to a cell and the height corresponds to the slope. The distinct gap between the bottom cells and top cells for each chip justify that cell balancing is working when hard coding the values. We then moved to the actual cell balancing logic and saw the voltage differential between the max cell and the min cell go from $\sim 20\text{mV}$ at the start of the test to $<5\text{mV}$ where the threshold to stop balancing was 5mV , justifying that cell balancing was working.

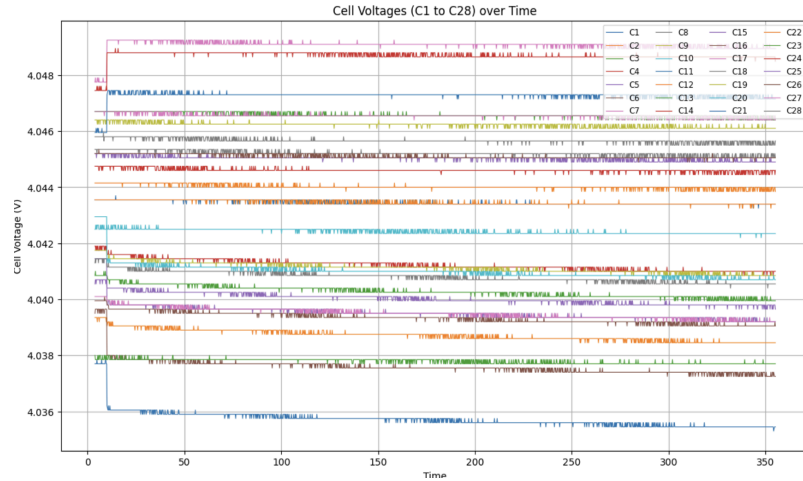


Figure 41: Segment 1 Cell Voltages Over Time with Cell Balancing Working

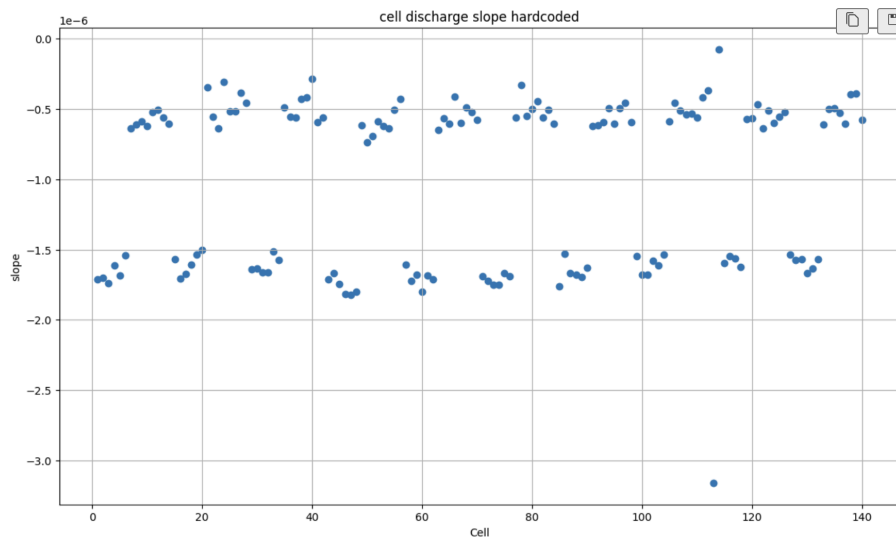


Figure 42: Segment 1 Cell Voltage Slopes with Cell Balancing Working

5.5 Open Wire Check

This is another small section, but worth documenting how we validated open wire check. We have a test battery that is 14 cells in series with their voltages broken off. This goes to a connector that plugs into a simple PCB that takes these voltages and routes them to the same connectors as the sense PCBs with the same spacing. The idea is the daughterboards and flex PCBs can then be plugged in to simulate a segment. We do this so that we can test on lithium, not just voltage dividers, before installing the daughterboards on segments. To perform OWC we instead plug the flex PCBs into a breadboard and use jumper wires to connect to this test board.

The idea is that we can unplug an individual jumper wire and validate that an open wire is read. We then repeat this process of unplugging jumper wires to test all 14 channels per chip and do this for both chips on each daughterboard.

5.6 Charging with the Elcon Charger

The Elcon 6.6kW air cooled charger we have is used by many other FSAE and FS teams. We have found that it is really easy and intuitive to use. The charger just sends a single CAN message with the voltage present at its terminals, the current, and the fault statuses, and it only expects one CAN message with the max voltage, max current, and whether or not to enable charging. The only strange-ish behavior is it will current limit up until the max voltage and then when it starts to voltage limit it bumps up the voltage to 1V higher than the max voltage. It does this because it needs to be at a slightly higher voltage than the cells to actually shove charge into them. It also is due to when current stops flowing the cells will relax voltage back down. This relaxation is very significant and non-trivial as shown by the dip when charging stops around 3700 seconds in Figure 43.

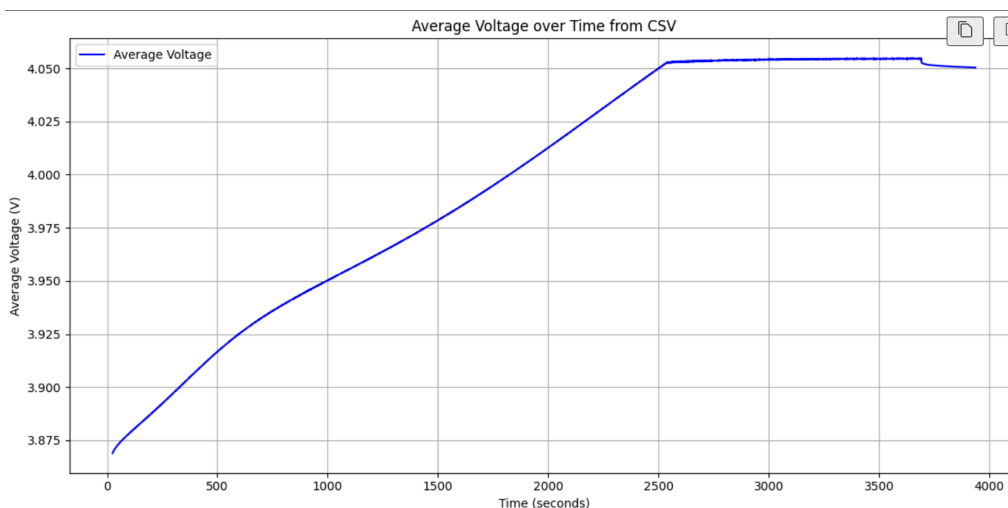


Figure 43: Average Voltage During a Charging Cycle

For our use cases though it doesn't even make sense to even charge to >99% or anywhere near 1V of the absolute max voltage. We actually started to see some issues of bumping into overvoltage when doing dynamic testing at high SOC's, but more on that later.

5.7 Dynamic Testing

When we finally got the car on jacks and the battery at high SOC's we noticed a significant issue. The max and min voltage would stride away from the average voltage/stationary voltage. This would present as an overvoltage at high SOC's where the max voltage increasing would cause an overvoltage. After many days of testing and trying different solutions, it was discovered that this was not happening to every cell, but just two next to each other, such that C126 would decrease and C127 would increase by the same amount. Looking into this more the flex PCB had a cold solder joint that was not making a proper connection. After fixing this there was no more min-max diverge away from the average during dynamic testing. This was not discovered by the open wire check because that only runs on the S-channel and this joint was on the C-channel. We have since added open wire check to the C-channel following the same ideology as the S-channel, only replacing the S registers with C ones.

There was another interesting dynamic overvoltage that we have also observed. When spinning the motor on the jacks, and regen disabled, the voltages of every cell would increase when quickly letting off the accelerator. At a first glance this looked fine, but it would increase past the open circuit voltage which doesn't make much sense. The only way for this to happen is if current is being sent backwards into the battery by the inverter, but regen is disabled. Either way, that is what must be happening because there also reads a current backwards supporting the behavior observed by the voltages. Looking online it is not problematic for the cells to briefly go above the max voltage as long as it isn't for a long time, and they relax back down. To enable this we adopted an alert maturation criteria. This is the same philosophy that OEMs use, such that an overvoltage event is allowed as long as it is under a short period of time, such that the event has to mature before it gets alerted. After consulting with the data available for similar cells, it was determined to set the maturation time to be 2 seconds with a hard overvoltage of 4.35V. This is derived from multiple sources (EV manufacturers) citing running their li-ion at up to 4.35V for brief periods of time. It is also worth testing with the on-chip C-channel filters to see if they would be able to help smooth out any sharp spikes in voltage.

While we have yet to test regen at high SOC's, it is worth noting that not even EV manufacturers have figured out how to do this and simply just do not perform regen braking at high SOC's.

5.8 Chips Dying

This is the largest question mark up in the air at the end of this year. The ADBMS6830B chips seemingly die occasionally when opening/closing the contactors. This has now been observed 4 times. Three of which happened on opening the contactors and the same chip was dying, the last chip in the stack. This is really quite strange as the failure mode is the first ADC fries and shorts to ground allowing $\sim 150\text{mA}$. This is also strange because it only happens to the same chip and only on opening the contactors. It is very unclear why this happens as not only the chips, but the boards have been changed out, and it has still killed a chip. A couple things to note is that it has only happened at $>95\%$ SOC and the first voltages on the last chip are where the longest traces are on the sense PCBs.

We have some theories as to what is causing this, namely going outside the max/min voltage of the ADC noted absolute rating section of the datasheet. The hard part is it is not clear why it only happens very rarely and only on contactors changing state. There was not enough time with competition quickly approaching this year, but the first thing to do when we get back is to scope at that ADC and see if it gets near the absolute rating/what it looks like. Again it is hard to think of what is causing this because it happens so infrequently. We just need more information. For now, the solution to limp us through comp is replace a chip if it dies. This is an acceptable solution as the chips only appear to die on contactors and not during driving.

This is something that needs to be thoroughly looked into in the coming years and one, if not the most, pressing issue.

6 Future Designs

6.1 RTOS and DMA

As mentioned above, it would be very useful to have a constant stream of data, communicating with the daughterboards via DMA. This would then allow for the other tasks to happen at the same time. This is very different from this BMS as currently it gets the raw voltages and temp from the BMS, then transforms them into useful values, then updates states, then sends out CAN and USB messages each loop set with a timer of 200ms, occasionally performing OWC every 30 seconds. Unfortunately this follows very closely to the bare metal OS philosophy. This has its own pros and cons, but again the BMS would greatly benefit from having the daughterboard values being gathered in the background with an RTOS scheduling the other tasks.

This should be doable to add next year, because we already have our own drivers and just need to adjust them to support interrupt or DMA. The RTOS should also be straightforward to add as there was some work put into adapting the code when it was using their drivers to have an RTOS. It didn't really provide any tangible benefits as the majority of the time is spent waiting for the SPI polling, but the groundwork has been laid for when we are not polling.

6.2 Pack Monitor

This is the other major change to be made apart from the ones listed in the list of improvements. The current measurement currently has LSB of $\sim 0.1\text{A}$ and a noise floor higher than that. On top of that there is currently no way to read the total voltage, only aggregate the individual cells, and there is no way to read the voltage on the other sides of the contactors during precharging, and as such we have to rely on the inverter voltage. All these issues plus more quality of life things like pyro-fuse control capability are present in the third chip in the sponsored AD chips, the ADBMS2950. The nomenclature for this chip is the pack monitor as it does high level monitoring of the pack. The most important function in this chip is the current measuring. It isn't worth getting into in this report but there are multiple differential ADCs that are highly tuned to work best on reading the voltage across a shunt resistor. There are two 24 bit $\pm 130\text{mV}$ range

differential ADCs providing significant improvements than the current measurement currently being done on the mainboard.

This chip is already being looked into and integrated at a high level into the design in the hopes of adding it quickly to the BMS at the start of next year.

6.3 Having the BMS as purely a slave

This has been talked about in depth in section 4.2, but the summary is that having the BMS have states and serve as more than a slave is unproductive when there is already an electric control unit (ECU) serving as the brains of the car. For context, inverters and COTS BMSs all interact explicitly as slaves, and it is not ideal that we are doing something different.

References

- [1] Analog Devices, “ADBMS6822 Datasheet and Product Info | Analog Devices.” Accessed: Feb. 06, 2025. [Online]. Available: <https://www.analog.com/en/products/adbms6822.html>

- [2] “ADBMS6830B Datasheet and Product Info | Analog Devices.” Accessed: Mar. 18, 2025. [Online]. Available: <https://www.analog.com/en/products/adbms6830b.html>

- [3] SAE International, “Formula SAE Electric.” Accessed: Feb. 11, 2025. [Online]. Available: <https://www.sae.org/site/attend/student-events/formula-sae-electric>

- [4] SAE International, “Formula SAE Rules 2025.” Aug. 31, 2024. [Online]. Available: <https://www.fsaeonline.com/cdsweb/gen/DownloadDocument.aspx?DocumentID=4bafd174-62da-48b6-b016-589385ca5ae6>

- [5] Molicel, “INR-21700-P45B Datasheet.” [Online]. Available: https://www.molicel.com/wp-content/uploads/INR21700P45B_1.2_Product-Data-Sheet-of-INR-21700-P45B-80109.pdf

- [6] Ewert Energy Systems, “Orion BMS 2 Specifications.” [Online]. Available: https://www.orionbms.com/downloads/documents/orionbms2_specifications.pdf

- [7] R. Suganya, L. M. I. L. Joseph, and S. Kollem, “Understanding lithium-ion battery management systems in electric vehicles: Environmental and health impacts, comparative study, and future trends: A review,” *Results in Engineering*, vol. 24, p. 103047, Dec. 2024, doi: 10.1016/j.rineng.2024.103047.